

線形代数演算ライブラリBLASとLAPACKの 基礎と実践 (II) MPLAPACKについて (高精度BLAS, LAPACK)

中田 真秀

理化学研究所 開拓研究本部 柚木研究室

2023-06-01 計算科学技術特論A

BLAS, LAPACK高精度計算編 講義内容

1. MPLAPACKの開発に影響を与えた考え方:富豪的プログラミング
2. 私は如何にして心配するのをやめて高精度LAPACKを作るようになったのか (半正定値計画法)
3. コンピュータにおける(高精度)浮動小数点数演算
4. MPLAPACKの誕生 (旧名MPACK)
5. 8年の停滞のあとまさかのブレイクスルー！MPLAPACK 2.0.1のリリース
+ベンチマーク結果
6. LAPACK (MPLAPACK)のテストは何をしてるのか。
7. MPLAPACKを使ってみる
8. 巨大なライブラリの書き直しで何を学んだか。

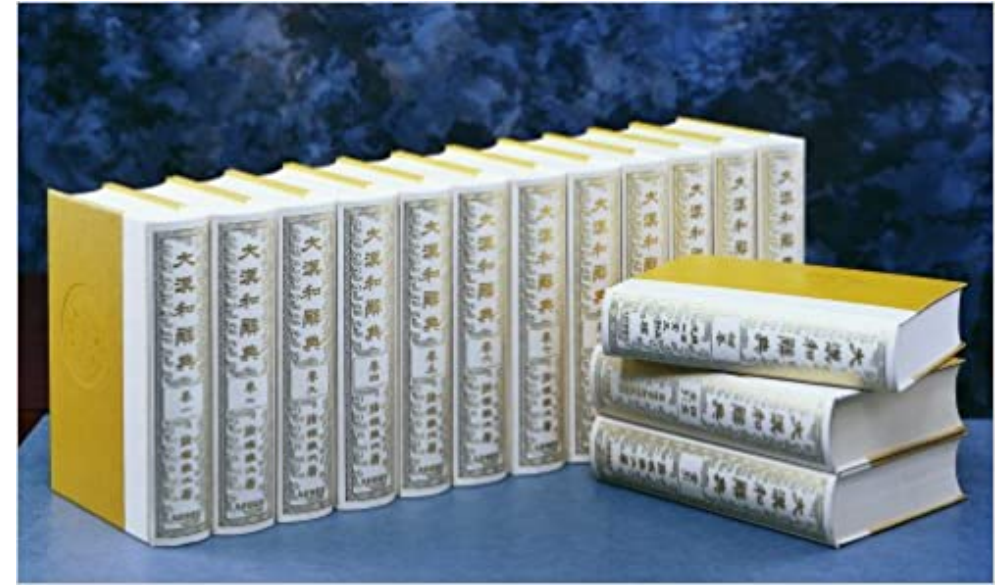
MPLAPACK開発に 影響を与えた考え方

富豪的プログラミング

- <http://www.pitecan.com/fugo.html>
- 「メモリや実行効率を気にしないでお気楽にプログラムを作る」
- 「効率を重視したプログラムは作るのが大変ですし、ちゃんと動かすにはデバッグも大変です。 富豪的プログラミングでは一番単純で短いアルゴリズムを使います。」
- <https://web.archive.org/web/20030902021235/http://www.pitecan.com/fugo.html>
- 初出2003年9月頃？ 増井俊之による
- 知ったのは2006年頃か

諸橋轍次の執念

- 大漢和辞典で有名、以下序文より
- 東洋の文化は、その大半が漢字漢語によって表現せられている。それは文芸に於ても思想に於ても、将た又道德宗教に於ても皆然りである。それ故、漢字漢語の研究を外にして東洋の文化を云為することは不可能である
- 二十年二月二十五日の劫火によって一切の資料を焼失した。
- 以前に用いた活字は既述の如く全部焼失したので、これを改めて木版に彫り、更に活字を作るとすれば、少なくとも十年二十年の歳月を要する。
- 過去十年殆ど失明同様の状態にあった私は、幸今春、名医の手術によって隻眼を開いた。



コロンブスの卵

- 自明なことをやってみたかった
- ぶっちゃけLAPACKをC++に変えただけやん
- ちがうちがうちがう

数値線形代数に新しい価値を加える

私は如何にして心配するのを
やめて高精度LAPACKを作
るようになったのか

半正定値計画法の紹介

$$\begin{array}{ll} \text{Primal SDP:} & \text{minimize} \quad \sum_{i=1}^m c_i x_i \\ & \text{subject to} \quad X = \sum_{i=1}^m F_i x_i - F_0, \\ & \quad \quad \quad X \succeq O \\ \text{Dual SDP:} & \text{maximize} \quad F_0 \bullet Y \\ & \text{subject to} \quad F_i \bullet Y = c_i \quad (i = 1, 2, \dots, m), \\ & \quad \quad \quad Y \succeq O. \end{array}$$

primalとdualという等価な問題がある。

X, Y は半正定値(固有値が0以上)の行列。半正定値を保ったまま線形関数の最小値を行う

最適化条件は、 $XY=0$ (X, Y 少なくともどちらかは条件数が発散)

特徴:

primal は最適解の上界、dualは最適解の下界を出し、最適になると一致

Max-cut (NP-完全の問題)に対して0.878近似を与える(Williamson-Goeman)

内点法で効率的に解ける -> 藤澤らのSDPAが実装では最高速

量子化学から半正定値計画問題へ

- 量子化学の基底状態のエネルギーなどは縮約密度行列を変分すれば求まる。 $E_g = \min_{\Gamma^2 \in N\text{-rep.}} \hat{H}\Gamma^2$
- 中田ら J. Chem. Phys 2001 (<https://aip.scitation.org/doi/10.1063/1.1360199>) で、これが**半正定値計画問題**に帰着できて、実際にSDPAというソルバで説いて、分子、原子でかなり良い結果を得た。

Variational calculations of fermion second-order reduced density matrices by semidefinite programming algorithm

J. Chem. Phys. 114, 8282 (2001); <https://doi.org/10.1063/1.1360199>

Maho Nakata, Hiroshi Nakatsuji, and Masahiro Ehara

• Department of Synthetic Chemistry and Biological Chemistry, Graduate School of Engineering, Kyoto University, Kyoto 606-8501, Japan

Mitsuhiro Fukuda

more...

色々悩んだ挙げ句(2000年代前半)

- 量子化学的には有効数字10桁くらいほしい。あと1-2桁くらい足りない
 - 最適解付近で行列の条件数が発散するため、解を精度良く求めるのは難しい。

多倍長精度計算が必要

色々悩んだ挙げ句(2000年代前半)

- 量子化学的には有効数字10桁くらいほしい。あと1-2桁くらい足りない
 - 最適解付近で行列の条件数が発散するため、解を精度良く求めるのは難しい。
- 半正定値計画ソルバーSDPA(藤澤ら)を使っていた。
 - BLAS, LAPACKを沢山使っていた。プログラムはC++で書かれており、かなり複雑
 - 初期バージョンはmesarchを使っておりBLAS, LAPACKを用いていなかった。
 - 精度は8桁で十分とみな思っていた。工場の在庫が1万個と1万1個に差はない。
- 精度保証は2000年代前半にはあまり発達しておらず
 - LU分解など部品のいくつかは精度保証はできたが、全部やるのは途方もなく感じた。
- 多倍長精度による計算は時間かかるが、それをやるかかなと。
- すでにBNCpackという幸谷 智紀先生による多倍長精度の線形代数ライブラリがあった。
 - CでGMP/MPFRを直に叩いていた。
 - SDPAはC++で複雑。BNCpackを使うように書き直すのは複雑だしバグが混入しそうでイヤ

高精度半正定値計画ソルバ SDPA-GMPの開発を行った

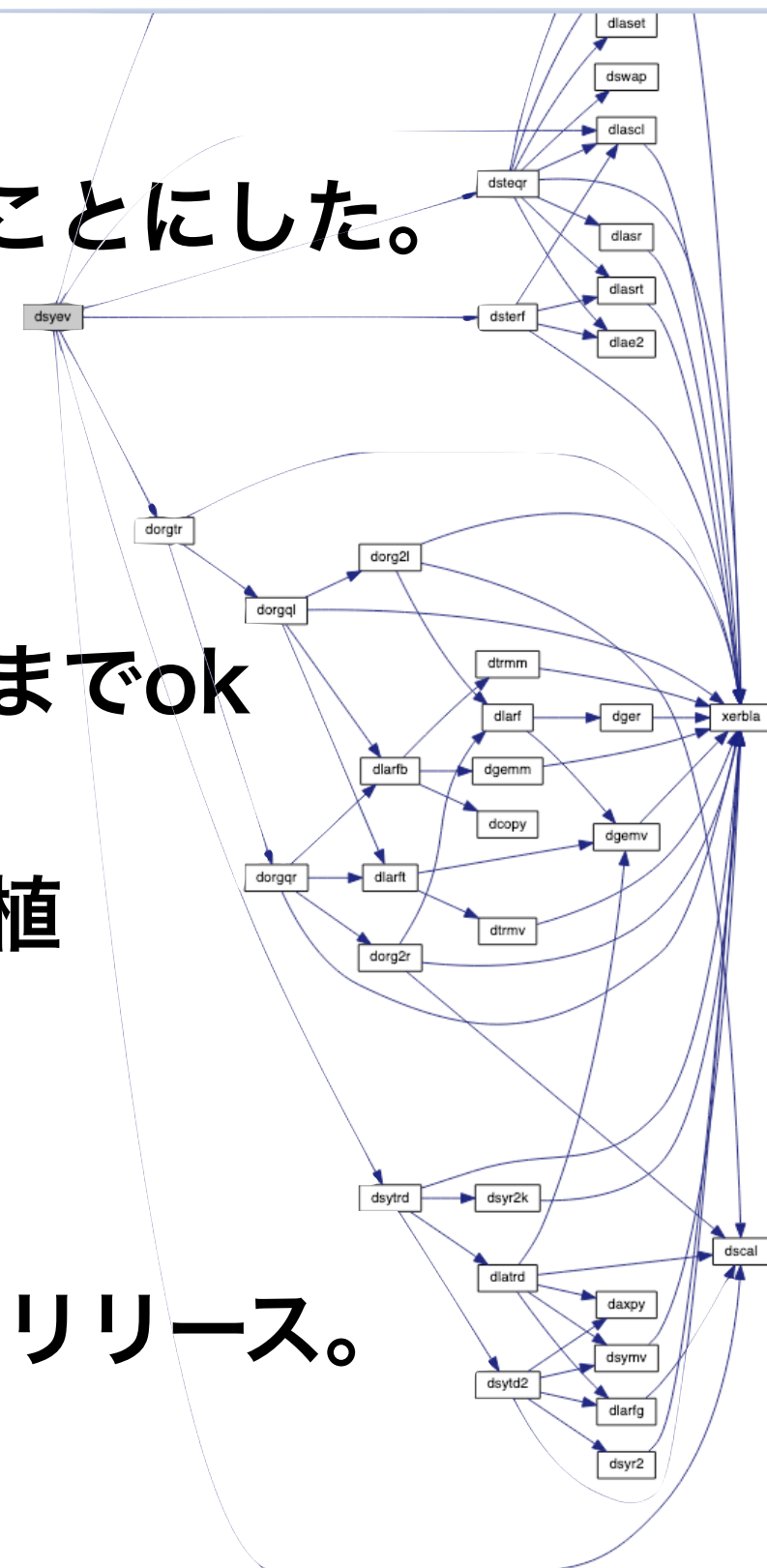
- 富豪的プログラミング: C++でdoubleの感覚で多倍長精度計算をすべし。
 - SDPAはC++で書かれていた。
- 確実に解があるといいたい。
- 細くても(遅くても)切れる(高い精度の)プロ用の道具がほしい。

日本人的に陥りやすい罠(私見)

- 富豪的プログラミングと諸橋轍次は中田に衝撃を与えた。
- 日本人は超絶技巧が好き
 - 発展的な、複雑な理論、計算手法開発。
- 日本は常にリソースが不足している
 - 1bitでも少なく、1clockでも速く、1pJでも消費電力が少なくしたい。
- パラダイムシフトや、コロンブス卵的研究は多数派ではない。
 - **BLAS, LAPACKのようなAPIを定められない**
 - BLAS, LAPACKは科学に大きな影響を与えたプログラムの一つ。
 - <https://www.nature.com/articles/d41586-021-00075-2>
- 線形代数は自分が今思っているよりはるかに重要なんじゃないか？と思い始めた。
 - 人類の歴史に古くから存在する。

MPLAPACK前史

- BLAS, LAPACKをなるべくそのまま移植することにした。
- SDPAに使われている関数を実装していった。
- コレスキー分解、対称行列の固有値を求めるまでok
- **dsyev**までたどり着くまで… 50個くらい移植
- <https://github.com/nakatamaho/sdpa-gmp/tree/master/>
- GNUやオープンソースを信奉してたのですぐリリース。



結果

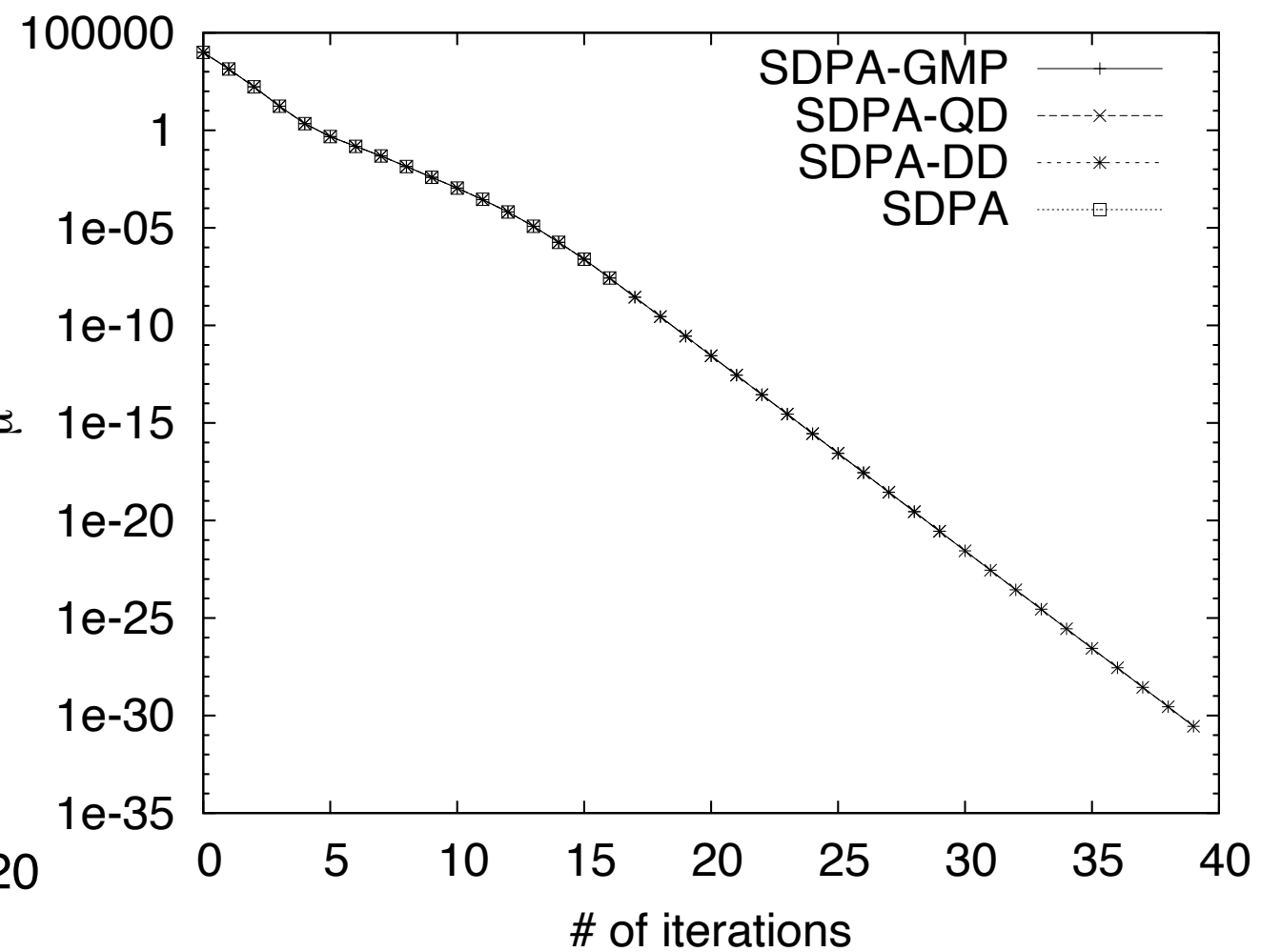
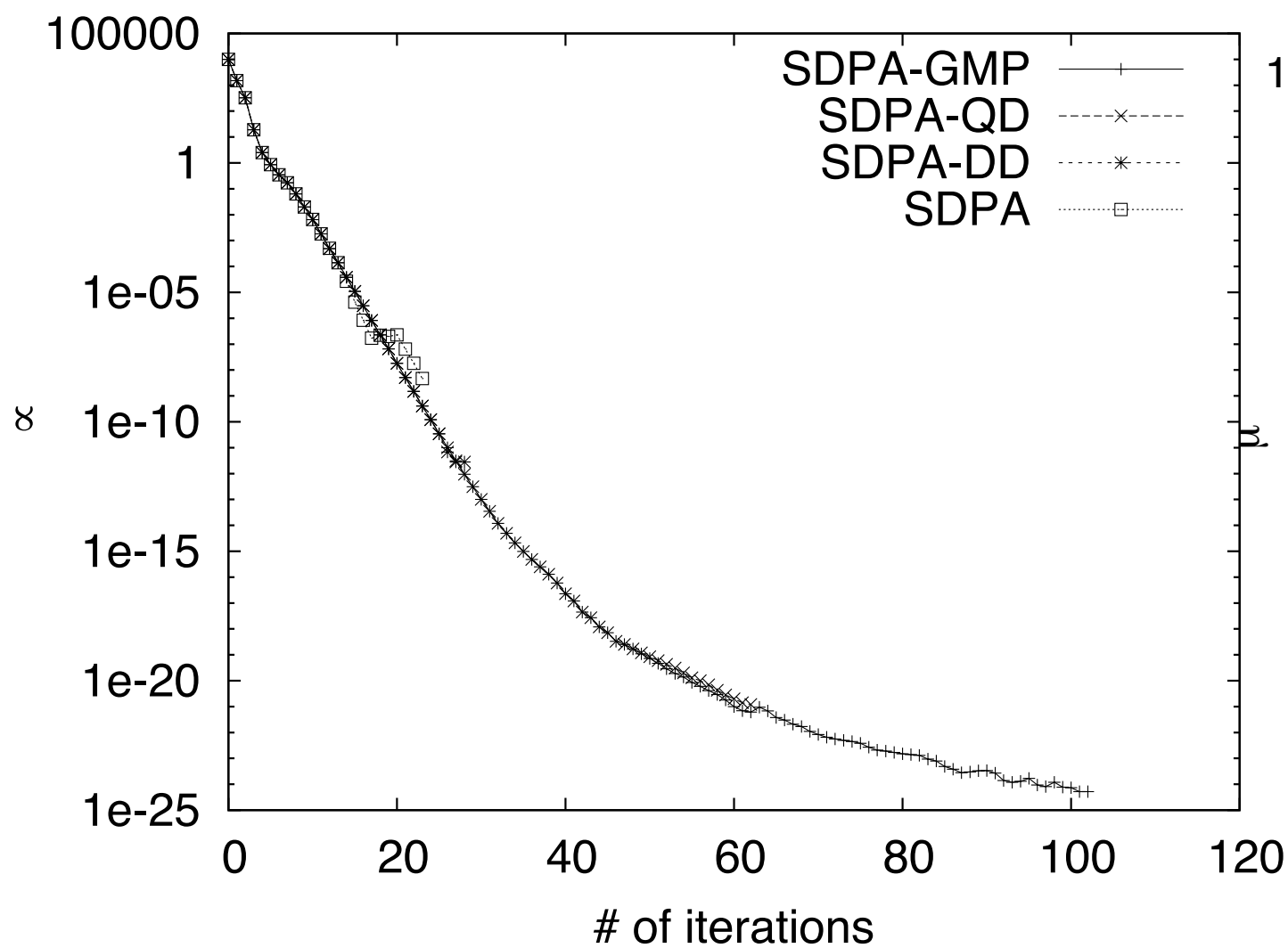
- J. Chem. Phys. 2008
 - Hubbard modelで $U/t \rightarrow \infty$ でちゃんと値が出てきた。
- SDPの問題で不安定、最適解がわかってない問題へも応用できた。
- <https://ieeexplore.ieee.org/document/5612693>
 - 81回も引用していただき感謝 (2023-05-24; google scholar)

A numerical evaluation of highly accurate multiple-precision arithmetic version of semidefinite programming solver: SDPA-GMP, -QD and -DD.

SDPの収束の様子

Gpp250 内点がないill conditionな問題

maxG11 (Slater条件を満足)



高精度半正定値計画法の応用例

- 三次元イジングモデル; 共形場理論を用いた結果が厳密解と等しい予想がある。

- <https://arxiv.org/pdf/1406.4858.pdf> : SDPA-GMPを用いている

The existence of only two relevant scalars is an obvious experimental fact about the 3D Ising CFT — it follows from the observation that the phase diagram of water is two-dimensional. Nevertheless, despite the mild assumptions of $\mathbb{Z}_2$ symmetry and two relevant scalars, we find a striking result: the dimensions $(\Delta_\sigma, \Delta_\epsilon)$ are almost uniquely fixed! The allowed region is a tiny sliver around $\Delta_\sigma = 0.51820(14)$ and $\Delta_\epsilon = 1.4127(11)$, in agreement with the most precise Monte-Carlo simulations [35], and also the c -minimization conjecture semidefinite program solver. We give details of our implementation in the solver **SDPA-GMP** [47, 48] in appendix B.

<https://arxiv.org/pdf/1502.02033.pdf> 独自ソルバ

The analysis of [9] did not use permutation symmetry of the OPE coefficient $\lambda_{\sigma\sigma\epsilon} = \lambda_{\sigma\epsilon\sigma}$.¹⁴ Including this constraint leads to an additional modest reduction in the allowed region,¹⁵ which we plot in figure 2 at $\Lambda = 43$. The resulting island gives a rigorous determination $\Delta_\sigma = 0.518151(6)$, $\Delta_\epsilon = 1.41264(6)$, which is 5-10 times more precise than the Monte Carlo results of [67]. We summarize the comparison to Monte Carlo in figure 3.

- SDPB is approximately 3500 lines of C++. It uses the GNU Multiprecision Library (GMP) [62] for arbitrary precision arithmetic, and MPACK [63] for arbitrary precision linear algebra. The relevant MPACK source files are included with SDPB, with some slight modifications:

SDPの精度保証

VSDP: Verified SemiDefinite-quadratic-linear Programming

Version 2018

C. Jansson, M. Lange, and K. T. Ohlhus

VSDP is a software package for the computation of verified results in conic programming. It supports the constraint cone consisting of the product of semidefinite cones, second-order cones, and the nonnegative orthant. It provides functions for computing rigorous error bounds of the true optimal value, verified epsilon-optimal solutions, and verified certificates of infeasibility. All rounding errors due to floating-point arithmetic are taken into account.

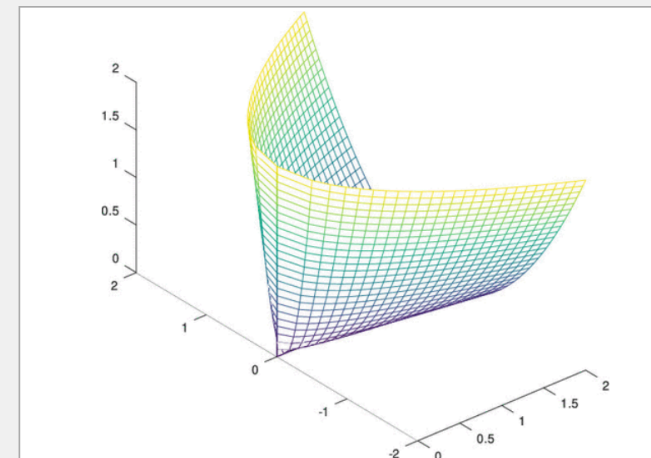
Rigorous Lower Bounds for the Ground State Energy of Molecules by Employing Necessary N-Representability Conditions

Denis Chaykin, Christian Jansson*, Frerich Keil*, Marko Lange*, Kai Torben Ohlhus*, and Siegfried M. Rump*

Abstract

Electronic structure calculations, in particular the computation of the ground state energy, lead to challenging problems in optimization. These problems are of enormous importance in quantum chemistry for calculations of properties of solids and molecules. Minimization methods for computing the ground state energy can be developed by employing a variational approach, where the second-order reduced density matrix defines the variable. This concept leads to large-scale semidefinite programming problems that provide a lower bound for the ground state energy. Upper bounds of the ground state energy can be calculated for example with the Hartree–Fock method or numerically more exact for a given basis set by full CI. However, Nakata et al. (*J. Chem.*

*Phys.*200111482828292) observed that due to numerical errors the semidefinite solver produced erroneous results with a lower bound significantly larger than the full CI energy. For the LiH, CH⁻, NH⁻, OH, OH⁻, and HF molecules violations within one mhartree were observed. We applied the software VSDP which takes all numerical errors due to floating-point arithmetic operations into consideration. For two test libraries VSDP provides tight rigorous error bounds lower than full CI energies reported with an accuracy of 0.1 to 0.01 mhartree. Only little computation work must be spent in order to compute close rigorous error bounds for the ground state energy.

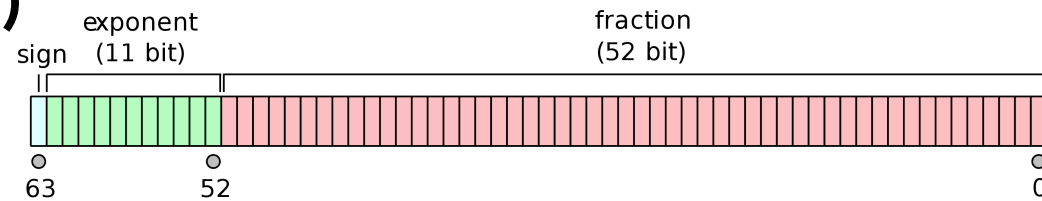


J. Chem. Phys. 2001の我々の結果、誤差が大きい場合があったことも検証してくれた

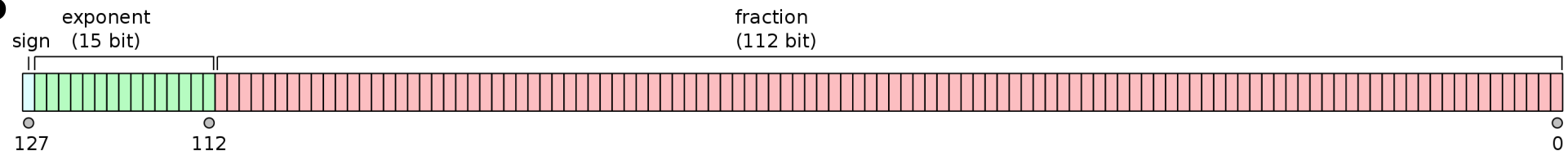
コンピュータにおける (高精度)浮動小数点数演算

浮動小数点演算と多倍長精度演算ライブラリ

- binary64(倍精度)

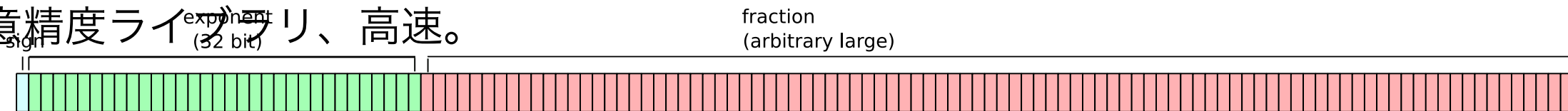


Binary128



- 精度が必要な問題に対しては、単に仮数部を増やして力づくで解く方法を採用。

- GMP: 任意精度ライブラリ、高速。

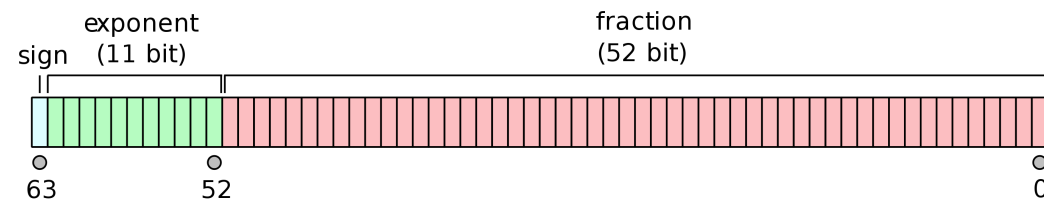


- MPFR: GMP+IEEE754ライクな丸めつき、つまりより誠実な演算をする。
- MPC: MPFRに基づく複素数演算ライブラリ。初等関数も実装されている。
- DD, QD: 倍精度変数を複数もつことでほぼ4, 8倍精度を実現。いわゆるIBM方式

ユーザのニーズに合わせて、高精度演算ライブラリはいくつも存在する

binary64

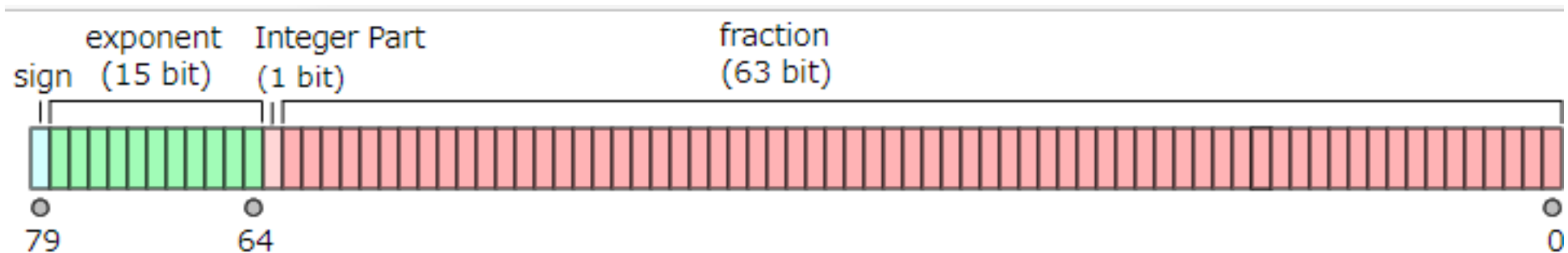
- binary64(倍精度) 10進数16桁程度



- 64bit=8bytesで1つの実数を表現する
- いわゆる「倍精度」
- もっともポピュラーな浮動小数点数。ほぼすべてのプロセッサでハードウェアのサポートがあるため高速。

_Float64x

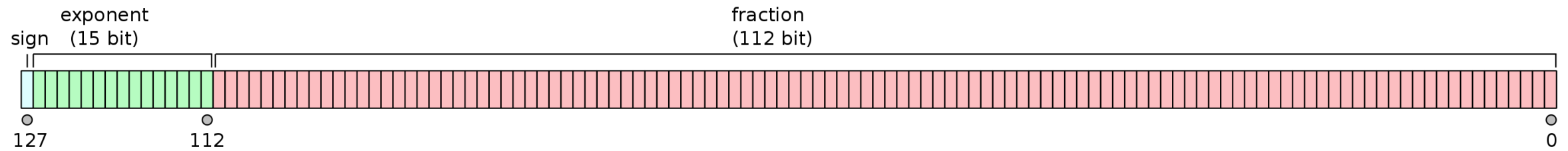
- _Float64x : 80bit浮動小数点数。IEEE 754 2008で規格外になった
- CではTS 18661-3で(今更)初めてサポートができた。C++のサポートはなし(GNU拡張はあり)
- (拡張倍精度) 10進数19桁程度



- 80bit=8bytesで1つの実数を表現する; 128bitで格納するか? 80bitで格納するか?
- いわゆる「拡張倍精度」 exponentのrangeがbinary64より大きい。
- x87/68881でサポートされている/いた浮動小数点数。もはやSSEやAVXが主流なのでほぼ使われてない。後方互換のためだけに存在する。遅い。
- 32bitのx86ではx87を使っていてlong doubleが_Float64であったこともあった。
- Kahan(IEEE 754の父)が導入した。大変不評→倍精度にするためには二回丸めが必要→値が変わることがある→他のbinary64のみとの互換性が取れない→大変不評(他にも色々)

cf. https://qiita.com/mod_poppo/items/9588b6f425ffe4b5c7bf

binary128



- binary128はIEEE 754 2008で定義された
- 10進数33桁ほどの精度
- いわゆる四倍精度。
- ハードウェアサポートはIBM z13程度。他はすべてソフトウェア実装。非常に遅くなる。
- gccなどでは利用できるが、過渡期のようにサポート状況は大変混とんとしている。

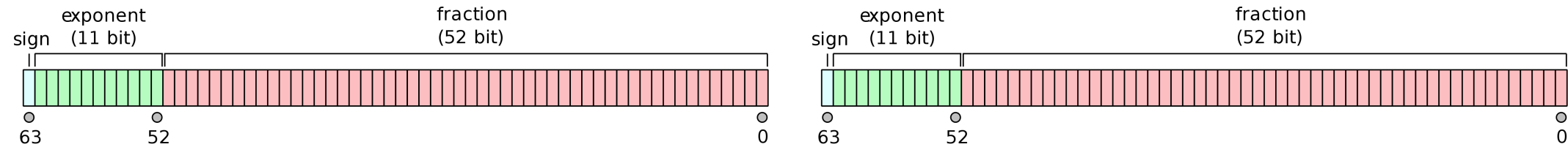
binary128の状況は混沌としている

- binary128はIEEE 754-2008で規格化され、gcc4.3 (2009年頃) から `__float128` という形で使えている (GNU拡張)
- Gcc10.xでもaarch64のように `__float128/libquadmath` が使えない環境はある。わざと潰してある。
- Cでは、TS 18661-3でbinary128の実装である、`_Float128` が定義された。
 - C++ではまだ何も定まっていない(!) が、GNU拡張で `_Float128` が使える場合もある。
- libquadmathやglibcのサポートは必須。printfや、sin, cos...など。
- 少なくとも6種類存在。
 - long doubleのみ存在そしてこれはbinary128。 `__float128`, `_Float128` は使えない (CentOS7, 富岳 AArch64)
 - `_Float128` はgcc/glibcでサポートされていて、long doubleと同じ (CentOS8 AArch64)
 - `_Float128` はgccで `__float128` とlibquadmathとして存在している (MacOS x86_64, mingw64)
 - `_Float128` はglibcでサポートされていて、binary128型は一つしか利用できない (Intel C/C++ 多分バグ)
 - `_Float128` はgccとglibcでサポートされていて、`_Float128`, `__float128` 両方使える (Ubuntu20.04, amd64)
 - `_Float128` はgccとglibcでサポートされていて、`_Float128` のみ使える (Ubuntu22.04, amd64)

https://qiita.com/mod_poppo/items/8a61bdcc44d8afb5caed

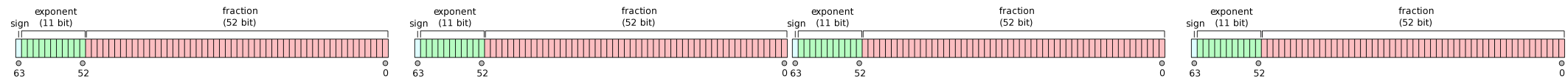
は大変参考にした。

倍々精度



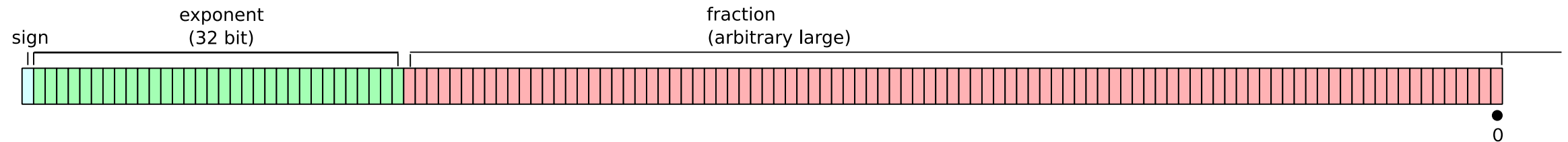
- binary64をふたつ並べて10進32桁ほどの精度を実現
- Knuth-Dekkerの方法を使うと無誤差でbinary64の数同士の足し算、掛け算ができる。
- IBM方式ともいわれ、IBMのマシンやコンパイラ、PowerMac、PowerPCのgccのlong doubleは倍々精度である/あった(obsolated)
- Y. HidaらのQD libraryを用いた。
- doubleだけで計算できるので高速。(数10GFlops-TFlopsは出せる)
- exponentが削れるため、倍精度で表せる数が倍々精度で表せない場合がある。

8倍精度



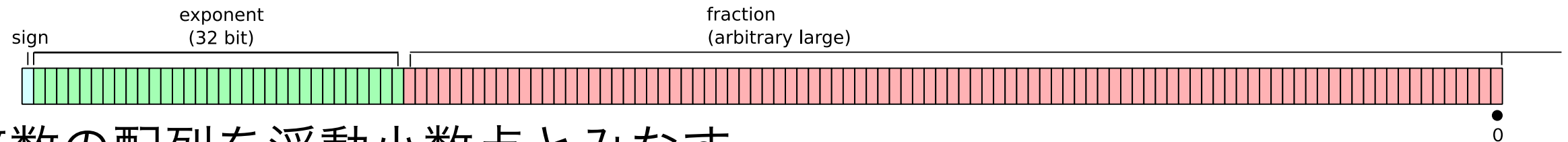
- binary64を4つ並べて10進64桁ほどの精度を実現
- Knuth-Dekkerの方法を使うと無誤差でbinary64の数同士の足し算、掛け算ができる。
- Y. HidaらのQD libraryを用いた。
- doubleだけで計算できるが、GMPなどにそろそろ速度的にコンパラになる。

The GNU Multiple Precision Arithmetic Library



- 整数の配列を浮動小数点とみなす。
- 速度に主眼を置いており、fractionは64bit単位でないと増やせない、丸めモードがない、exponentのrangeが固定という制約がある。
- sqrtはあるが、初等関数(sin, cos, tan, log, exp, arctan...)などが**ない**
 - MPLAPCKで一部利用している。今のところbinary64のそれを利用しつつお茶を濁している
- 複素数はないので自作(C++の勉強したかっただけ)

The GNU MPFR Library



- 整数の配列を浮動小数点とみなす。
- GMPをベースに、IEEE 754ライクな丸めモードを搭載+exponentが可変 (精度を意図的に低くもできる)。真面目に計算するときに重要。
- 初等関数(sin, cos, tan, log, exp, arctan, sqrt...)などがある
- 複素数ライブラリのMPCも存在する(MPLAPACKで利用)
- GMPと比べると少し(かなり)遅い。

MPLAPACKの誕生

(旧名MPACK)

線形代数: 現行BLAS, LAPACKではAPI定義、規模、スピードという価値のみ。精度という新しい価値を加えたい。

- 50個BLAS, LAPACKの関数を実装していったら、どうせならBLASは全部、LAPACKもいくつかやってみてもいいじゃないか。できるんじゃないかと思い始めた。
- BLAS, LAPACKは素直で綺麗なコード
- FORTRAN77からCに書き直すのは良いことなんではないか？
- 線形代数は歴史を紐解くと長いし、重要度はますます増すばかり。
- 精度という価値はあんまり重要視されてない。線形代数に一つ、貢献できるかも。

MPLAPACK (IMPACT)へ

- 多倍長精度のBLAS, LAPACKはちゃんと作っておこう。
 - C++の勉強を兼ねて使ってみた。
- 体力気力がどこまで続くかやってみようと思ってやってみた。BLASはすぐ終わるが、LAPACKは見えない。完璧を目指すより何かを出すことを目標。
- 開発方針
 - C++で書いて、doubleやfloatと同じ感覚で使えるようにすること。ただし、C++は便利なCであること。C++は難しい...
 - FORTRAN77から脱却し、よりモダンな設計にしたい。
 - なるべく多くの多倍長精度ライブラリを取り込むこと。
 - 標準化と、実装の最適化を分けること。
 - SDPAとはリンクできること。実用例が重要。

プログラミングモデル

- 関数の名前はPrefixを変えた。“R” や “C”を用いた。あとは小文字。
 - daxpy -> Raxpy, zgemm -> Cgemm
- INTEGER, REAL, COMPLEXという仮想的な型を用いて実装した。
- C++のクラスを使ってREALなどを実現。
- typedefを用いて REAL -> mpf_class, qd_real, __float128など実際の型を指定した。
 - テンプレートは用いず。
 - Cgemm<mpf_real, mpf_complex>はカッコ悪い
 - C++も進化してきたしテンプレート版作成予定
- 初等関数sin, cos, log, exp…などはあれば使うが、なければdoubleで代用。

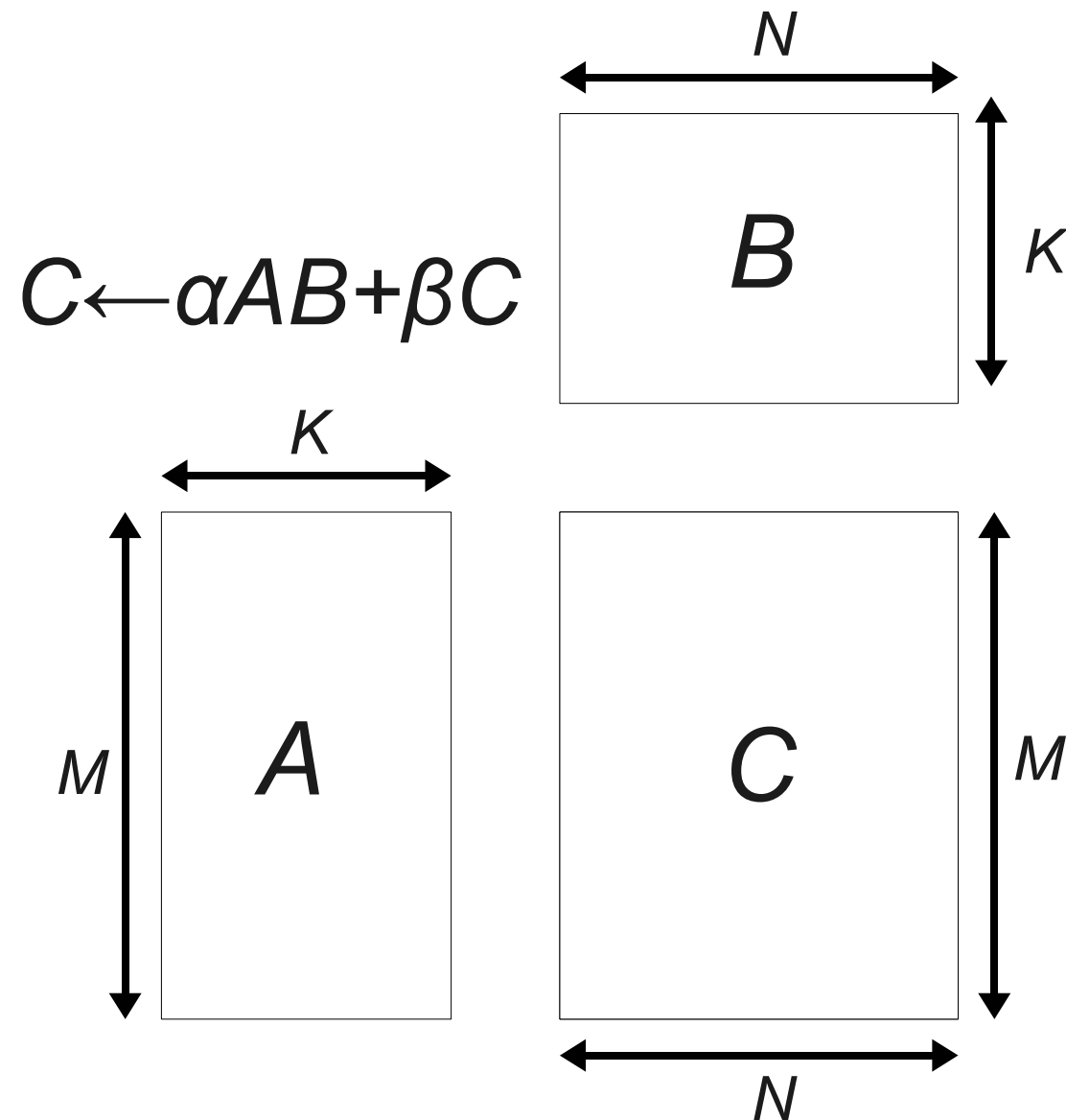
実装方法

- BLAS, LAPACKをf2cを用いてCのコードに変換
 - f2cは完全なCのコードを出力する。ほぼ100%変換できる。しかし人間が読めるコードにはならない。
 - 初期の頃は手探り状態で、目で見ながら変換してた。
- Sedをヘビーに使って修正
 - C++に変換。
 - パーサに手を入れようとしたが殆ど読めず。yacc, lexなどを学んだことなく、理解できず。
- ひたすら手で直す。
- ランダムな値を入れてBLAS, LAPACKのルーチンと比較。差が小さければok

FORTRANとC++の違いの吸収方法

- 配列は0から始まる(C++) 1から始まる (FORTRAN)
 - 二次元配列がC++にはないので、leading dimensionを陽に用いて一次元配列に開いた。どのルーチンにも必ずleading dimensionの記述はある。
 - 一次元配列はA(I) -> a(i-1)と1引くことにした。
 - 二次元配列はa(i,j) -> a[(i-1) + (j-1)*lda]と変換した。
- Do loop は 1から始まり、Nで終わることが多い。
- Do loopは+1/-1されるかが分からない場合がある。以下のイディオムが役に立った。
 - for (i=p; inc>0 ? i<=n : i>=n ; i=i+inc) {
- FORTRANはcall by referenceなので、かならずCではポインタ渡しになる。
- COMMON, FORMATもWRITEもBLAS, LAPACKには存在しなかった。
 - これはいいこと。I/Oを使わない!!助かった
 - ただしテストプログラムでFORMAT, WRITEは多用している。
 - GOTOは残した。しかし手でcontinue, breakなどにしたこともあった。
- 初期の変換は目で行っていて「自然な」ループ (for (i=0; i<n; i++))にしていたこともある。

dgemmの変換を見てみよう



BLAS: dgemmの抜粋

```
IF (NOTA) THEN
*
*      Form  C := alpha*A*B + beta*C.
*
      DO 90 J = 1,N
        IF (BETA.EQ.ZERO) THEN
          DO 50 I = 1,M
            C(I,J) = ZERO
50          CONTINUE
        ELSE IF (BETA.NE.ONE) THEN
          DO 60 I = 1,M
            C(I,J) = BETA*C(I,J)
60          CONTINUE
        END IF
        DO 80 L = 1,K
          TEMP = ALPHA*B(L,J)
          DO 70 I = 1,M
            C(I,J) = C(I,J) + TEMP*A(I,L)
70          CONTINUE
80          CONTINUE
90          CONTINUE
      ELSE
```

dgemmをf2cに通した場合

```
if (nota) {  
  
/*          Form  C := alpha*A*B + beta*C. */  
  
    i__1 = *n;  
    for (j = 1; j <= i__1; ++j) {  
        if (*beta == 0.) {  
            i__2 = *m;  
            for (i__ = 1; i__ <= i__2; ++i__) {  
                c__[i__ + j * c_dim1] = 0.;  
            }  
        }  
        /* L50: */  
        } else if (*beta != 1.) {  
            i__2 = *m;  
            for (i__ = 1; i__ <= i__2; ++i__) {  
                c__[i__ + j * c_dim1] = *beta * c__[i__ + j * c_dim1];  
            }  
        }  
        /* L60: */  
        }  
        i__2 = *k;  
        for (l = 1; l <= i__2; ++l) {  
            if (b[l + j * b_dim1] != 0.) {  
                temp = *alpha * b[l + j * b_dim1];  
                i__3 = *m;  
                for (i__ = 1; i__ <= i__3; ++i__) {  
                    c__[i__ + j * c_dim1] += temp * a[i__ + l *  
                        a_dim1];  
                }  
            }  
        }  
        /* L70: */  
        }  
        /* L80: */  
    }
```

***n, *betaなどはcall by referenceだから**

アンダースコア二つ入れるのが醜い

これを「自然な」C++にしたい...

Rgemm(旧版)

```
152 //Start the operations.
153     if (notb) {
154         if (nota) {
155             //Form C := alpha*A*B + beta*C.
156             for (mpackint j = 0; j < n; j++) {
157                 if (beta == Zero) {
158                     for (mpackint i = 0; i < m; i++) {
159                         C[i + j * ldc] = Zero;
160                     }
161                 } else if (beta != One) {
162                     for (mpackint i = 0; i < m; i++) {
163                         C[i + j * ldc] = beta * C[i + j * ldc];
164                     }
165                 }
166                 for (mpackint l = 0; l < k; l++) {
167                     if (B[l + j * ldb] != Zero) {
168                         temp = alpha * B[l + j * ldb];
169                         for (mpackint i = 0; i < m; i++) {
170                             C[i + j * ldc] =
171                                 C[i + j * ldc] + temp * A[i + l * lda];
172                         }
173                     }
174                 }
175             }
176         } else {
```

C++らしくゼロからループを始めてた

多倍長精度Rgemm(現在)

バグ入らないようにC++/Fortranを摺り寄せた

自動変換!

```
if (nota) {  
    //  
    //      Form  C := alpha*A*B + beta*C.  
    //  
    for (j = 1; j <= n; j = j + 1) {  
        if (beta == zero) {  
            for (i = 1; i <= m; i = i + 1) {  
                c[(i - 1) + (j - 1) * ldc] = zero;  
            }  
        } else if (beta != one) {  
            for (i = 1; i <= m; i = i + 1) {  
                c[(i - 1) + (j - 1) * ldc] = beta * c[(i - 1) + (j - 1) * ldc];  
            }  
        }  
        for (l = 1; l <= k; l = l + 1) {  
            temp = alpha * b[(l - 1) + (j - 1) * ldb];  
            for (i = 1; i <= m; i = i + 1) {  
                c[(i - 1) + (j - 1) * ldc] += temp * a[(i - 1) + (l - 1) * lda];  
            }  
        }  
    }  
}
```


「正しい値がでてきているか」 のチェック

- どうしたら正しい値が出てくるか、のチェックは本質的に難しい。
- 「品質保証」 Quality assurance (QA) と呼ばれる
- まず、MPFR版とオリジナルのBLASに、ランダムな値を色々入れてみて、コードを隈なく通っているかどうかを確認しつつ、十分結果が近ければMPFR版は正しい、とした。
 - GMP版、QD版....はMPFR版と比較してより厳しい閾値でチェックした。
 - これは本質的にはQAではないが、BLAS, LAPACKは正しいことを根拠に、正しいことにした。研究での利用には耐えているので、いいと思う。
 - 時間もものすごくかかる。Rgemmだと、 n, m, k 以外に lda, ldb, ldc もあるので6重ループになる。さらにtransposeで4通り…と積み重なる。

Rgemmのチェックプログラム

```
void Rgemm_test3(const char *transa, const char *transb, REAL_REF alpha_ref, F
{
    int errorflag = FALSE;
    int mplaack_errno1, mplaack_errno2;
    for (int k = MIN_K; k < MAX_K; k++) {
        for (int n = MIN_N; n < MAX_N; n++) {
            for (int m = MIN_M; m < MAX_M; m++) {

                int minlda, minldb;

                if (Mlsame(transa, "N"))
                    minlda = max(1, m);
                else
                    minlda = max(1, k);

                if (Mlsame(transb, "N"))
                    minldb = max(1, k);
                else
                    minldb = max(1, n);

                for (int lda = minlda; lda < MAX_LDA; lda++) {
                    for (int ldb = minldb; ldb < MAX_LDB; ldb++) {
                        for (int ldc = max(1, m); ldc < MAX_LDC; ldc++) {

#ifdef VERBOSE_TEST
                            printf("#k is %d, n is %d, m is %d, lda is %d, ldb is %d, ldc is %d\n", k, n, m, lda, ldb, ldc);
                            printf("#transa is %s, transb is %s\n", transa, transb);
#endif
                        }
                    }
                }
            }
        }
    }
}
```

MPACK 0.8.0 (2012-12-25)

- 多倍長精度のBLAS, LAPACK
- API提供を目的
- MBLASはすべて実装、MPALACPKは100ルーチン実装。
 - 実対称、エルミート固有値問題、LU、コレスキー分解、逆行列、条件数推定対応
- Linux/BSD/Mac/Win対応
- GMP, MPFR, binary128, quad-double, double-double, double, long double 対応
 - Rgemm (double-double) はCUDAでC2050対応
 - Intel Xeon Phi対応
- C++で書かれていて、double/floatのようにプログラム可能。
- 2-条項BSDライセンス

MBLAS 76ルールチン

古いリリース

LEVEL1 MBLAS

Crotg	Cscal	Rrotg	Rrot	Rrotm	CRrot	Cswap
Rswap	CRscal	Rscal	Ccopy	Rcopy	Caxpy	Raxpy
Rdot	Cdotc	Cdotu	RCnrm2	Rnrm2	Rasum	iCasum
iRamax	RCabs1	Mlsame	Mxerbla			

古いリリース

LEVEL2 MBLAS

Cgemv	Rgemv	Cgbmv	Rgbmv	Chemv	Chbmv	Chpmv	Rsymv
Rsbmv	Ctrmv	Cgemv	Rgemv	Cgbmv	Rgemv	Chemv	Chbmv
Chpmv	Rsymv	Rsbmv	Rspmv	Ctrmv	Rtrmv	Ctbnv	Ctpmv
Rtpmv	Ctrsv	Rtrsv	Ctbsv	Rtbsv	Ctpsv	Rger	Cgeru
Cgerc	Cher	Chpr	Cher2	Chpr2	Rsyrr	Rspr	Rsyrr2
Rspr2							

古いリリース

LEVEL3 MBLAS

Cgemm	Rgemm	Csymm	Rsymm	Chemm	Csyrr	Rsyrr	Cherk
Csyrr2k	Rsyrr2k	Cher2k	Ctrmm	Rtrmm	Ctrsm	Rtrsm	



MLAPACK 100ルーチン

古いリリース

Mutils	Rlamch	Rlae2	Rlaev2	Claev2	Rlassq	Classq
Rlanst	Clanht	Rlansy	Clansy	Clanhe	Rlapy2	Rlarfg
Rlapy3	Rladiv	Cladiv	Clarfg	Rlartg	Clartg	Rlaset
Claset	Rlasr	Clasr	Rpotf2	Clacgv	Cpotf2	Rlascl
Clascl	Rlasrt	Rsytd2	Chetd2	Rstear	Csteqr	Rsterf
Rlarf	Clarf	Rorg2l	Cung2l	Rorg2r	Cung2r	Rlarft
Clarft	Rlarfb	Clarfb	Rorgqr	Cungqr	Rorgql	Cungql
Rlatrd	Clatrd	Rsytrd	Chetrd	Rorgtr	Cungtr	Rsyev
Cheev	Rpotrf	Cpotrf	Clacrm	Rtrti2	Ctrti2	Rtrtri
Ctrtri	Rgetf2	Cgetf2	Rlaswp	Claswp	Rgetrf	Cgetrf
Rgetri	Cgetri	Rgetrs	Cgetrs	Rgesv	Cgesv	Rtrtrs
Ctrtrs	Rlasyf	Clasyf	Clahef	Clacrt	Claesy	Crot
Cspmv	Cspr	Csymv	Csyr	iCmax1	RCsum1	Rpotrs
Rposv	Rgeequ	Rlatrs	Rlange	Raecon	Rlauu2	Rlauum
Rpotri	Rpocon					

古いリリース

**8年の停滞のあと
まさかのブレイクスルー
MPLAPACK 2.0.1のリリース!**

2012を最後にリリースが できなくなった。

- 燃え尽きた

- 高精度な非対称行列の固有値問題、特異値分解のルーチンはコミュニティから多数要望があったが、あまりの複雑さに音を上げてしまった。

- 品質保証をどうしたらいいかわからない

- 100個書いたが、これ以上書くモチベーションが無くなった。
- ランダムな値を入れて結果をLAPACKと比較して、という戦略だけでは、品質保証は難しい。
 - Rstegrなど、QR法で対角化する場合、かなり場合分けが多い。隈なくコードをチェックできているか？
 - 何をしているのか理解するのに時間がかかる、または分からない場合。

- 自分の興味に移ってきた。

- 機械学習の勃興により、久しぶりに量子化学をやり始めたら面白かった。
 - <https://nakatamaho.riken.jp/pubchemqc.riken.jp/>
 - 2021,2022と機械学習での予想コンテストKDD, NeurIPSでみんなに競っていただきました。
 - <https://ogb.stanford.edu/docs/lsc/pcqm4mv2/>

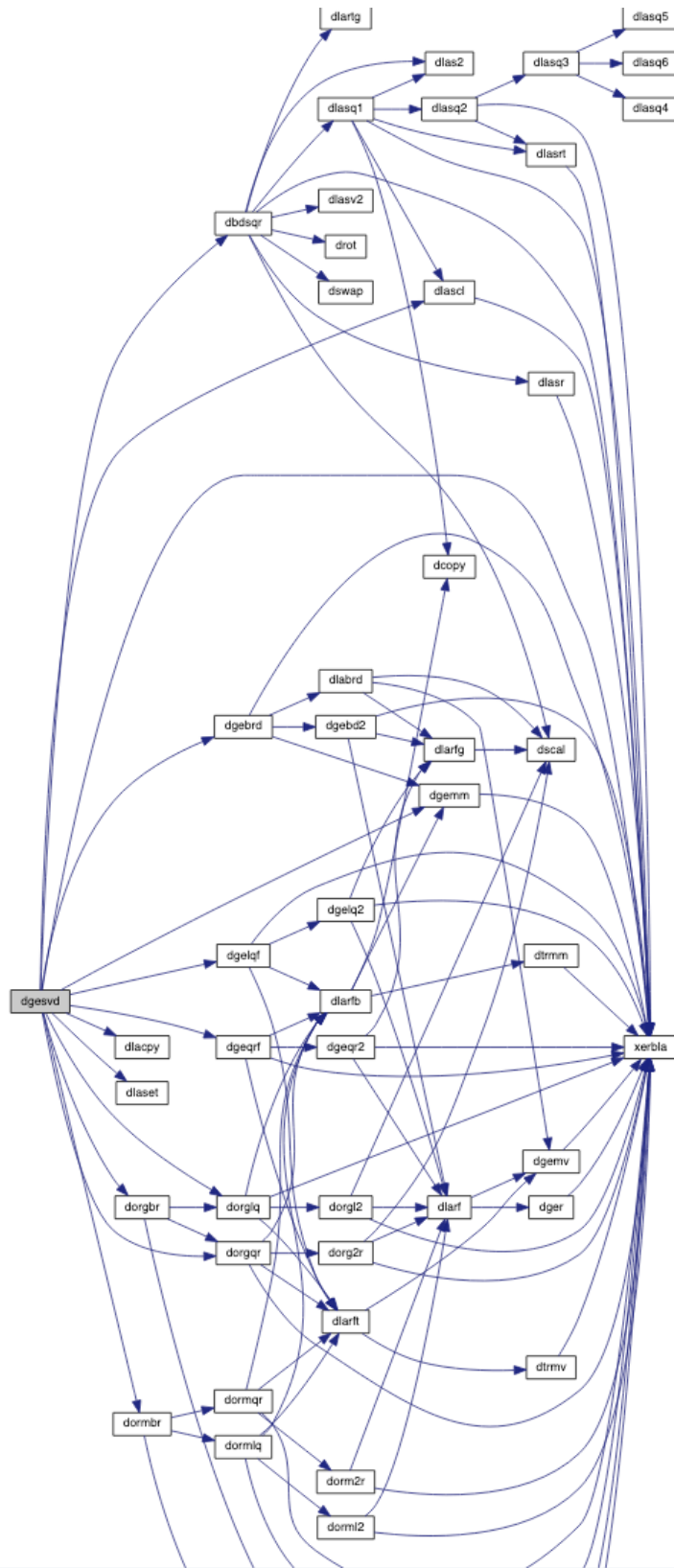
2012に残された課題

- mpackだと他のプログラムパッケージと名前がかぶっていて、mplapackに直したかった。
- F2cで変換したコードをもとに、手でC++に変換する、には限界があった。
 - Sedを使って直しても、可読性が低い。一日に変換できる量に限界があり、また、集中度によりバグがどう混入してるかわからない。
 - Cで書かれたパーサは書き直せなかった。出力されたコードの可読性は重要だとわかった。
- 実際の半正定値計画問題が解けるとかなり満足して、モチベーションが下がった。
- コード変換より、品質保証の手間が大きすぎた。
 - 600個程度あるLAPACKのルーチンのうち100個程度は変換できた。隈なくコードをなめるようなテストケースを書く必要がある。特にあるルーチンが理解できない場合、品質保証が難しい。
 - LAPACKのTESTING以下を移植するのは良いとはわかっていたが、FORTRAN+f2cでのformat文を手で直すのは困難。
- C++の変化が意外と激しくて、すぐエラーが出てくる。
- 開発環境の維持にコストが掛かりすぎる。
 - 仮想マシンも維持が結構めんどくさい。リソースも多くとられる。
 - OSやアーキテクチャはたくさんある。
 - IEEE 754-2008でbinary128が入ったが、コンパイラ、OSの状況は未だ混沌としている。

**特異値分解、非対称行列の固有値問題ソルバ
は特に非常に複雑**

dgesvd(特異値分解)のcall graph
dgesvdだけでも3000行以上あるのに
さらに30個以上の品質保証が必要。

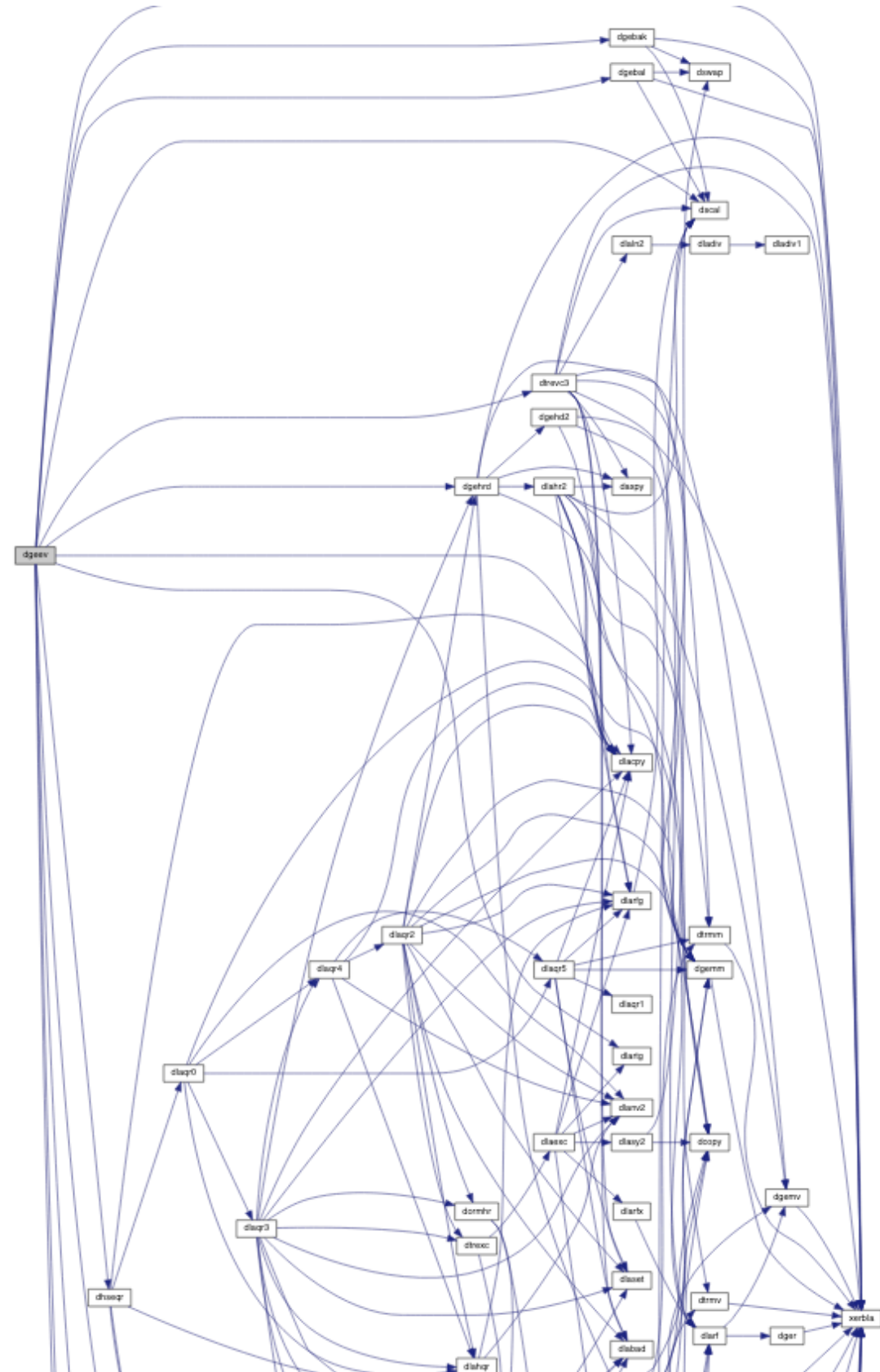
dgesvd(特異値分解)のcall graph
dgesvdだけでも3000行以上あるのに
さらに30個以上の品質保証が必要。



f2cと
気力だけでは
品質保証が
不可能

dgeev(非対称行列の対角化)のcall graph

dgesvdより長いし、依存する関数もより多い



**どうやってデバッグおよび
品質保証するか？**

その前にLAPACKは
どうやって品質保証してるか

LAPACKのTESTINGの移植

- LAPACKは自分の品質保証のためにTESTINGディレクトリが存在する。
 - LIN/xlintstd : binary64の実線形方程式ルーチンのテスター
 - LIN/xlintstz : binary64の複素線形方程式ルーチンのテスター
 - ELG/xeigtstd : 実固有値問題、特異値問題のテスター
 - ELG/xeigtstz : 複素固有値問題、特異値問題のテスター
 - MATGEN/ テスト用行列生成ライブラリ
- これらにxxx.inというファイルを食わせて正常終了したらok

LAPACKのテスト方法

- LAPACKをビルドする。
- lapack-3.xx.x/TESTING/ 以下にテスト用のファイルがある。
- `rm *out ; make 2>&1 | tee log` とするとテスト開始
- 🖱️テスト風景
- optimize levelを上げるとerrorも増える(ことがある)。

```
Testing REAL LAPACK linear equation routines
./LIN/xlintsts < stest.in > stest.out 2>&1
NEP: Testing Nonsymmetric Eigenvalue Problem routines
./EIG/xeigtsts < nep.in > snep.out 2>&1
SEP: Testing Symmetric Eigenvalue Problem routines
./EIG/xeigtsts < sep.in > ssep.out 2>&1
SEP: Testing Symmetric Eigenvalue Problem routines
./EIG/xeigtsts < se2.in > sse2.out 2>&1
SVD: Testing Singular Value Decomposition routines
./EIG/xeigtsts < svd.in > ssvd.out 2>&1
SEC: Testing REAL Eigen Condition Routines
./EIG/xeigtsts < sec.in > sec.out 2>&1
SEV: Testing REAL Nonsymmetric Eigenvalue Driver
./EIG/xeigtsts < sed.in > sed.out 2>&1
SGG: Testing REAL Nonsymmetric Generalized Eigenvalue Problem routines
./EIG/xeigtsts < sgg.in > sgg.out 2>&1
SGD: Testing REAL Nonsymmetric Generalized Eigenvalue Problem driver routines
./EIG/xeigtsts < sgd.in > sgd.out 2>&1
SSB: Testing REAL Symmetric Eigenvalue Problem routines
./EIG/xeigtsts < ssb.in > ssb.out 2>&1
SSG: Testing REAL Symmetric Generalized Eigenvalue Problem routines
./EIG/xeigtsts < ssg.in > ssg.out 2>&1
SGEBAL: Testing the balancing of a REAL general matrix
./EIG/xeigtsts < sbal.in > sbal.out 2>&1
SGEBAK: Testing the back transformation of a REAL balanced matrix
./EIG/xeigtsts < sbak.in > sbak.out 2>&1
SGGBAL: Testing the balancing of a pair of REAL general matrices
./EIG/xeigtsts < sgbal.in > sgbal.out 2>&1
SGGBAK: Testing the back transformation of a pair of REAL balanced matrices
./EIG/xeigtsts < sgbak.in > sgbak.out 2>&1
SBB: Testing banded Singular Value Decomposition routines
./EIG/xeigtsts < sbb.in > sbb.out 2>&1
GLM: Testing Generalized Linear Regression Model routines
./EIG/xeigtsts < glm.in > sglm.out 2>&1
GQR: Testing Generalized QR and RQ factorization routines
./EIG/xeigtsts < gqr.in > sgqr.out 2>&1
GSV: Testing Generalized Singular Value Decomposition routines
./EIG/xeigtsts < gsv.in > sgsv.out 2>&1
CSD: Testing CS Decomposition routines
```

LAPACKのテスト:サマリー+最適化

- gcc version 11.3.0 (Ubuntu 22.04, amd64)

```
--> LAPACK TESTING SUMMARY <--
Processing LAPACK Testing output found in the TESTING directory
SUMMARY          nb test run  numerical error  other error
=====
REAL              1317867      0      (0.000%)    0      (0.000%)
DOUBLE PRECISION  1318689      0      (0.000%)    0      (0.000%)
COMPLEX           777619      0      (0.000%)    0      (0.000%)
COMPLEX16         777338      1      (0.000%)    0      (0.000%)
--> ALL PRECISIONS  4191513      1      (0.000%)    0      (0.000%)
```

-O0 : 1個落ちてる

```
--> LAPACK TESTING SUMMARY <--
Processing LAPACK Testing output found in the TESTING directory
SUMMARY          nb test run  numerical error  other error
=====
REAL              1310091      1      (0.000%)    0      (0.000%)
DOUBLE PRECISION  1318689      0      (0.000%)    0      (0.000%)
COMPLEX           769843      1      (0.000%)    0      (0.000%)
COMPLEX16         778430      0      (0.000%)    0      (0.000%)
--> ALL PRECISIONS  4177053      2      (0.000%)    0      (0.000%)
```

-O2+ α : 2個落ちてる

```
--> LAPACK TESTING SUMMARY <--
Processing LAPACK Testing output found in the TESTING directory
SUMMARY          nb test run  numerical error  other error
=====
REAL              1296051     1303   (0.101%)    0      (0.000%)
DOUBLE PRECISION  1304649     1281   (0.098%)    0      (0.000%)
COMPLEX           747775     1224   (0.164%)    0      (0.000%)
COMPLEX16         768218      416   (0.054%)    0      (0.000%)
--> ALL PRECISIONS  4116693     4224   (0.103%)    0      (0.000%)
```

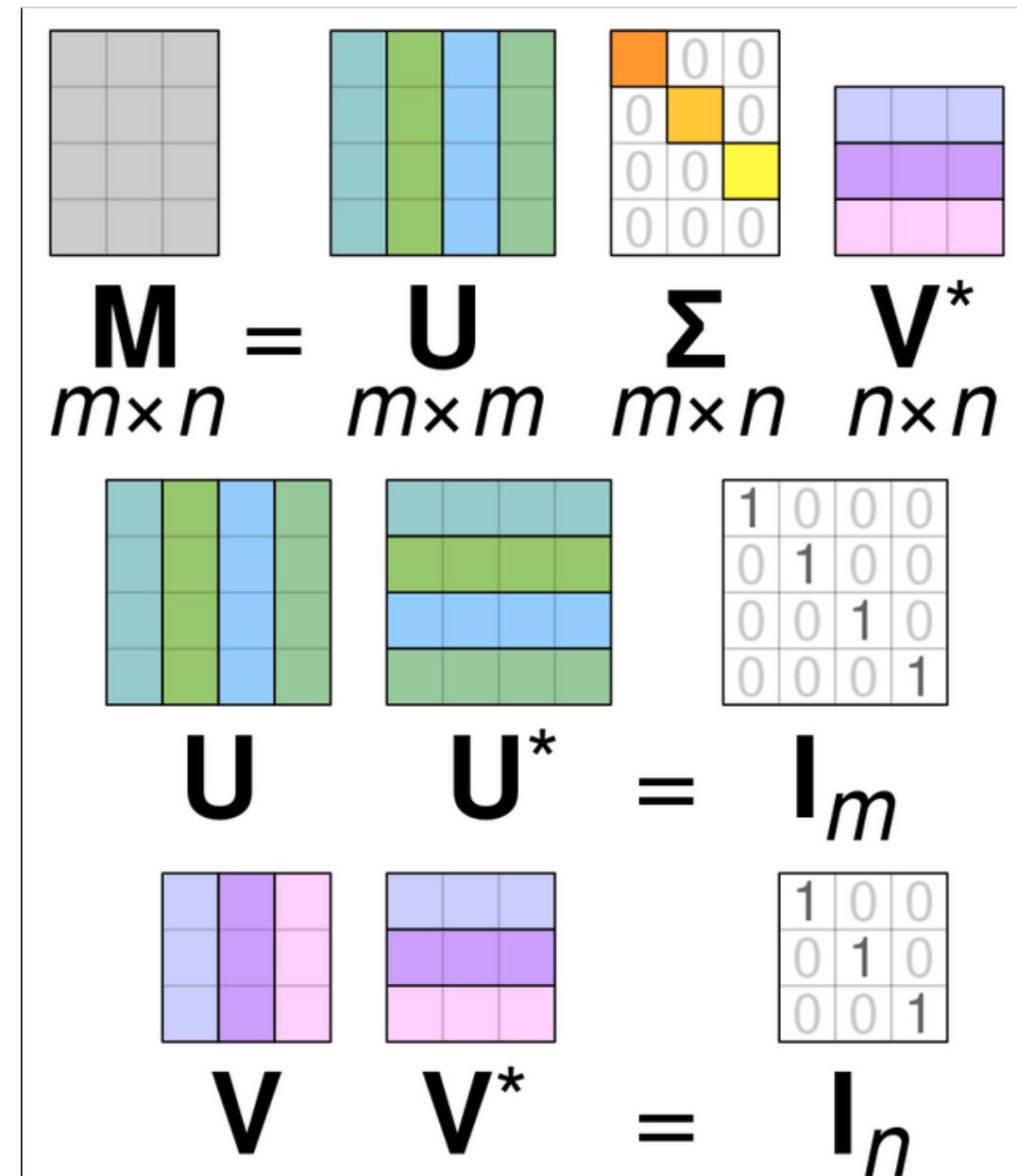
-O3+ α : 4224個落ちてる

LAPACKのテスト例 $Ax=b$

- $Ax=b$ (general linear equation solver), lapack-3.xx.y/TESTING/LIN/dchkge.f
 - DGETRF, -TRI, -TRS, -RFS, and -CONをテストする。
- 正解を先に作る。
 - A は様々な条件に変化させる。 A の条件数を、オーバーフローアンダーフロー付近までスケール、または2などにする。 A のノルムも同様に変化させる。
 - DGETRFならば、 A をLU分解するので、 $A_{\text{new}}=LU$ から $|A_{\text{new}} - A|$ が十分小さいならok 正確には $(\text{norm}(L*U - A) / (N * \text{norm}(A) * \text{EPS}))$,
 - つぎにDGETRIを使って A の逆行列を求める... ということを行う。
- 代数的な操作のみで解が求まるのでそこまで真面目にやらなくてよい。

LAPACKのテスト:SVD

- 特異値分解ではどうするか。
- lapack-3.xx.y/TESTING/EIG/ddrvbd.f
- $M=U\Sigma V^*$ で、まず Σ を作る。
 - 0, 1, ランダム行列、overflow付近までスケール、アンダーフロー付近までスケール
- $|A - U \text{diag}(S) V^T| / (|A| \max(M,N) \text{ulp})$
- $|I - U^H U| / (M \text{ulp})$
- $|I - V^T V^T| / (N \text{ulp})$
- $|I - U^H U| / (M \text{ulp})$
- $|U - U_{\text{partial}}| / (M \text{ulp}) |I - V^T V^T| / (N \text{ulp})$
- $|I - U^H U| / (M \text{ulp})$
- $|U - U_{\text{partial}}| / (M \text{ulp}) |I - V^T V^T| / (N \text{ulp})$
- $|I - U^H U| / (M \text{ulp})$
- ...これらが全部十分小さいか。DGESVD, DGESDD, DGESVDQ, DGESVDXなどいくつかルーチンがある



LAPACKのテスト:線形問題

- LAPACKビルド時に→のようにテストをする。
- xlintstd:binary64版線形方程式品質保証プログラム
- dtest.in : どのルーチンをどうテストするかが書かれていて、xlintstdに食わせる
- DGE: LU分解、逆行列、連立一次方程式などをチェックする
- DGB: バンド行列DGE
- DGT: 三角行列DGE
- TESTING/LIN/dchkaa.F

```
(fable38) docker@cd4e49cbd3ea:~/mplapack/external/lapack/work/internal/lapack-3.
xlintstd < dtest.in
Tests of the DOUBLE PRECISION LAPACK routines
LAPACK VERSION 3.*.1

The following parameter values will be used:
M   :      0      1      2      3      5     10     50
N   :      0      1      2      3      5     10     50
NRHS:      1      2     15
NB  :      1      3      3      3     20
NX  :      1      0      5      9      1
RANK:     30     50     90

Routines pass computational tests if test ratio is less than 30.00

Relative machine underflow is taken to be 0.222507-307
Relative machine overflow is taken to be 0.179769+309
Relative machine precision is taken to be 0.111022D-15

DGE routines passed the tests of the error exits
All tests for DGE routines passed the threshold ( 3653 tests run)
DGE drivers passed the tests of the error exits
All tests for DGE drivers passed the threshold ( 5748 tests run)
DGB routines passed the tests of the error exits
All tests for DGB routines passed the threshold ( 28938 tests run)
DGB drivers passed the tests of the error exits
All tests for DGB drivers passed the threshold ( 36567 tests run)
DGT routines passed the tests of the error exits
All tests for DGT routines passed the threshold ( 2694 tests run)
```

LAPACKのテスト:SVD

- ./EIG/xeigstd < svd.in
- xeigstd:binary64版固有値問題、特異値分解、など品質保証プログラム
- svd.in : 特異値分解品質保証のための入力ファイル
 - このような.inを23(実)+23(複素)計46個チェックする必要がある

Tests of the Singular Value Decomposition routines

LAPACK VERSION 3.*.1

The following parameter values will be used:

M:	0	0	0	1	1	1	2	2	3	3
	3	10	10	16	16	30	30	40	40	
N:	0	1	3	0	1	2	0	1	0	1
	3	10	16	10	16	30	40	30	40	
NB:	1	3	3	3	20					
NBMIN:	2	2	2	2	2					
NX:	1	0	5	9	1					
NS:	2	0	2	2	2					

Relative machine underflow is taken to be 0.222507-307

Relative machine overflow is taken to be 0.179769+309

Relative machine precision is taken to be 0.111022D-15

Routines pass computational tests if test ratio is less than 50.00

DBD routines passed the tests of the error exits (55 tests done)

DGESVD passed the tests of the error exits (8 tests done)

DGESDD passed the tests of the error exits (6 tests done)

DGEJSV passed the tests of the error exits (11 tests done)

DGESVDX passed the tests of the error exits (12 tests done)

DGESVDQ passed the tests of the error exits (11 tests done)

SVD: NB = 1, NBMIN = 2, NX = 1, NRHS = 2

All tests for DBD routines passed the threshold (10260 tests run)

All tests for DBD drivers passed the threshold (14820 tests run)

SVD: NB = 3, NBMIN = 2, NX = 0, NRHS = 0

LAPACKテストの問題点

- 非常に丁寧に作られているという前提で
- 固有値問題、特異値問題には収束の概念が入るので、原理的には代数的なテストプログラムではチェックできない
- ランダムな入力を用いているが...
 - seedを固定して乱数を発生させているためカバーできてるかどうか。
- コンパイラの最適化、実装の最適化をすると、誤差が大きくなる、またはテストに通らないことがある。
- テストはどのような入力の場合もテストできてるか？
 - 全てのルーチンのすべての場合分けを通っているか？
 - テスト行列の次元が50x50程度と比較的小さい

MPLAPACKでの苦闘

LAPACK/TESTINGのC++への移行

f2cでは事実上不可能

- BLASは代数的な演算だけなので、大雑把にBLASとMPBLASの値がだいたい合っていれば、実装がうまく行っているとは言える。
- LAPACKはどうすればいい？
 - LAPACKのTESTING/で良いテストを行っている。のでこれに乗っかりたい。
 - しかし、FORTRANのwrite, formatを手で直すのも事実上不可能。

f2c変換されたLAPACK/SRC/ TESTINGチェックルーチンの一部

```
/* Subroutine */ int aladhd_(integer *iounit, char *path)
{
    /* Format strings */
    static char fmt_9999[] = "(/1x,a3,\002 drivers:  General dense matrice"
        "s\002)";
    static char fmt_9989[] = "(4x,\0021. Diagonal\002,24x,\0027. Last n/2 co"
        "lumns zero\002,/4x,\0022. Upper triangular\002,16x,\0028. Random"
        ", CNDNUM = sqrt(0.1/EPS)\002,/4x,\0023. Lower triangular\002,16x,"
        "\0029. Random, CNDNUM = 0.1/EPS\002,/4x,\0024. Random, CNDNUM = 2"
        "\002,13x,\00210. Scaled near underflow\002,/4x,\0025. First colu"
        "mn zero\002,14x,\00211. Scaled near overflow\002,/4x,\0026. Last"
        " column zero\002)";
    static char fmt_9981[] = "(3x,i2,\002: norm( L * U - A ) / ( N * norm(A"
        ") * EPS )\002)";
    static char fmt_9980[] = "(3x,i2,\002: norm( B - A * X ) / \002,\002( n"
        "orm(A) * norm(X) * EPS )\002)";
    static char fmt_9979[] = "(3x,i2,\002: norm( X - XACT ) / \002,\002( n"
        "orm(XACT) * CNDNUM * EPS )\002)";
    static char fmt_9978[] = "(3x,i2,\002: norm( X - XACT ) / \002,\002( n"
        "orm(XACT) * (error bound) )\002)";
    static char fmt_9977[] = "(3x,i2,\002: (backward error) / EPS\002)";
    static char fmt_9976[] = "(3x,i2,\002: RCOND * CNDNUM - 1.0\002)";
    static char fmt_9972[] = "(3x,i2,\002: abs( WORK(1) - RPVGRW ) /\002,"
        "\002 ( max( WORK(1), RPVGRW ) * EPS )\002)";
```

課題の克服

- Fable: FORTRANからC++へ、可読性のあるコンバータの登場。
 - Pythonで書かれているため、パーサへの手入れが簡単。
 - 2012年頃はpython知らなかった。2015年ころから使い始めた。
 - FEM (Fortran EMulator) の搭載：**write, formatがほぼそのまま使えた。**
- Clang-format: C++の整形ツール。sedに渡しやすい。
- Docker: OSやアーキテクチャを手軽に試せた。
- Github: webベースの存在がありがたい。
- デパス: 鈍い頭痛、肩こりに20年位悩まされていたのが消えた。
- コロナ: 通勤しなくて良くなった。

fableの登場(2012)

- FORTRANからC++へのコードのコンバーター
- コードを生成するときの方針としては:
 - 厳密に同じ動きをさせる、が可読性が超低い(f2c)
 - 人間が読めて、たいていそのまま動く。さらに直す場合もなるべく手間が最小限(fable) … dsyevまではそのまま動くと論文にはあった。

<http://cci.lbl.gov/fable/>

fable - Automatic Fortran to C++ conversion

- **fable** converts fixed-format Fortran sources to C++. The generated C++ code is designed to be human-readable and suitable for further development. In many cases it can be compiled and run without manual intervention.
- fable comes with the C++ **fem** Fortran EMulation library. The entire fem library is inlined and therefore very easy to use: simply add `-I/actual/path/fable` or similar to the compilation command.
- The fem library has no dependencies other than a standard C++ compiler.
- fable is written in Python (version 2.x) which comes pre-installed with most modern Unix systems. A full Python distribution is included in the fable bundle for Windows.
- The name "fable" is short for "Fortran ABLEitung". "Ableitung" is a german word which can mean both "derivative" and "branching off".
- If you use fable in your work please cite:
Grosse-Kunstleve RW, Terwilliger TC, Sauter NK, Adams PD: *Automatic Fortran to C++ conversion with FABLE*. [Source Code for Biology and Medicine 2012, 7:5](#).
- fable and fem development was supported by supplemental funding from the American Recovery and Reinvestment Act (ARRA) to NIH/NIGMS grant number P01GM063210. The work was also supported in part by the US Department of Energy under Contract No. DE-AC02-05CH11231.
- fable and fem are a part of the cctbx open source project: [\[License\]](#) [\[Copyright\]](#)
- The fable Fortran reader could be re-used to generate code for other target languages.
- Send questions and comments to: fable@cci.lbl.gov

fableの登場(2012)

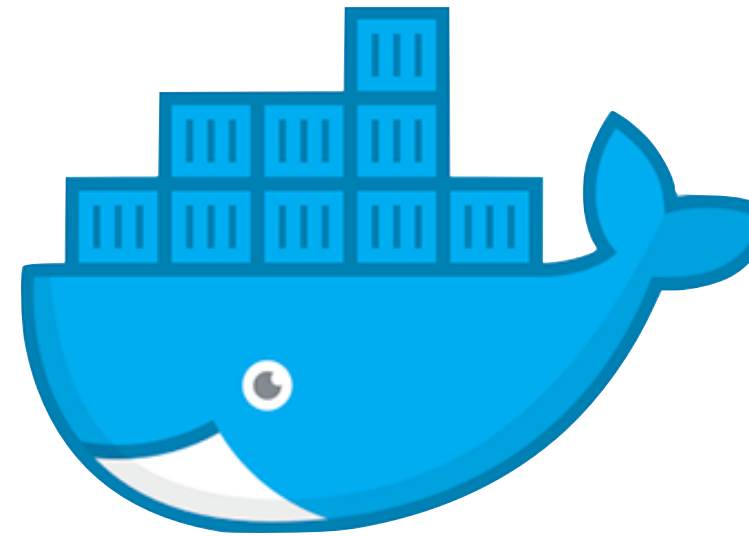
- Fortran -> C++ への変換部分はすべてpythonで書かれているため、可読性が高い。
 - 気に入らない変換があれば、直接手に入れられる。ライブラリも多く、正規表現も使える。
- FEM (fortran emulation library)
 - I/Oやintrinsic(内関数)はコンバートではなくライブラリに。
 - **LAPACKのTESTINGでwrite/format文を処理することが可能になった**

```
write(6, "(a)"), "ab";  
write(6, "(2a)"), "cd", "ef";  
write(6, "(a3)"), "gh";  
//Cierr write(6, '(a3.1)') 'ij'  
write(6, "(2a3)"), "kl", "mn";  
//C  
//Cgerr write(6, '(d)') 2.0  
//Cgerr write(6, '(2d)') 3.0, 4.0  
//Cierr write(6, '(d8)') 5.0  
write(6, "(d9.2)"), 6.0f;  
write(6, "(2d9.2)"), 7.0f, 8.0f;  
//C  
//Cgerr write(6, '(e)') 2.0  
//Cgerr write(6, '(2e)') 3.0, 4.0  
//Cierr write(6, '(e8)') 5.0  
write(6, "(e9.2)"), 6.0f;
```

←まさかのC++のコード!

Dockerの登場

- 仮想技術を使ったLinuxコンテナを作れる。
- OSなどの違いを吸収するのは意外と面倒。
- 仮想マシンだと、管理が必須。リソースも多く消費する。
- Dockerだと軽量なので気軽に環境の構築、破棄が可能。
 - 気が向いたときに環境構築だけに時間を費やして時間切れ、はもう嫌だ。
 - 他の人と成功、失敗を共有しやすい。再現度が高くなる
 - pythonの環境は無茶苦茶なので、仮想マシンを割り当てるのが妥当。
- Binary128のサポートは過渡期にあり、OSにより随分違うため試せる環境が必要。
 - PowerPC64le (IBM Power9?)を普通のインテルマシンで試せる



8年の停滞とv1.0.0への道のり

- FableでLAPACKのTESTINGをC++に移植できるかも!!
- デバスでプログラミング後の頭痛の解消
- もしかしたら今回は非対称行列の固有値問題や、特異値分解を実装できるかもしれないと思い、2021-03くらいから開発を再開してみた。
- <https://github.com/nakatamaho/mplapack/tree/v1.0.0>

開発の再開

開発方針

- Fableに手を加えてなるべく自動変換を行う
- BLAS, LAPACKともに自動変換で対処
- できないところはparserに手を加えたり、sedで対処
- 最後に手で直す

MPLAPACK特有の手入れ

- dconjg, dsqrtなどのトークンを後にsedで変換できるように文字列挿入。
- 文字列をFEM特有からcharの配列に直した。
- カッコをつけたり外したり、は正規表現で対処した部分もある。
- 複素数の宣言 `COMPLEX a = COMPLEX(0, 1);`のように初期化対応。
- 二次元配列、多次元配列は、fableでは関数になってた。これらはすべて一次元配列に直した。
 - leading dimensionは決め打ち(!)
 - `a(i,j) -> a[(i-1) + (j -1)* lda]` にパーサの段階で変換。
- DO LOOPはforにする。
 - Incrementが正、負が分かる場合は+1, -1 に。
 - Incrementが変数の場合はソースを読んで手で直す。
 - Incrementが正負とりうる場合は、イディオムを手で挿入(そんなに多くない)。

MPBLASはほぼ自動生成できるようになった。

- BLASは、C++、多倍長精度化も一箇所を除き自動で生成できるようになった!
- $\text{conj}(a) * a$ は実数だがこれは自動では変換できなかったなのでパッチを当てている。
- ヘッダも自動生成するようにした。

MPLAPACKでも、かなりの部分が自動的に変換できた

- さすがにMPLAPACKの全ては自動変換できなかった
- それでもLAPACK 3.9.1 の約1000個ルーチンのうち、約350個は自動変換できた(無修正でコンパイルできた)
- その他も軽微な修正で出来た
- 2021-04-04から2021-04-20までですべてのルーチンの手変換が終わった。
 - mpackのころは、何ヶ月もかかった。しかもできたルーチンは挙動不審。
 - 混合精度は今回はターゲットとせず(むしろしたほうがいいか)。
 - XBLAS利用による解の精度向上もターゲットとせず。

MPLAPACKの自動変換の 品質保証

- 2012年までの品質保証はまず行ってみた。
- LAPACKとMPLAPACKに同時にランダムに生成した行列を入れて、その値を比較するということを今回も行った。
- ほぼ無修正でテストにパスした。
- これまでのルーチンは実装は変わったが、使えることがわかった。

LAPACKのTESTINGの移植

- Fable + FEMで、format文はほぼそのまま使えることが判明したため、移植を試みた。
- 2021-05-12にとりあえず、
 - LIN/xlintst{R,C}_mpfr : MPFR版の実、複素線形方程式ルーチンのテスター
 - EIG/xlintst{R,C}_mpfr : MPFR版の実、複素固有値問題程式ルーチンのテスター

のビルドに成功。

LAPACKのTESTINGの移植

- ほぼ自動変換したプログラムのどこにバグが存在するか？
 - チェッカープログラムの移植不具合により適切に検出できてない場合がほとんど。
- それ以外は
 - maxlocの仕様がわかってなかった
 - Do loopで増減が両方の場合が存在(イディオムで対処)

探すのに苦労したバグ達

- 元のLAPACKのデバグにも役に立った
- アリエナイTESTINGの仕様
 - まさかのFortranで配列の添え字に0を使っていた(2か所)
- まさかの走ってないtestがあった。
 - githubでissue立てた→貢献者リストに名前が載った
 - <https://github.com/Reference-LAPACK/lapack/issues/563>
- テストのバグ; xlaenvの初期化忘れ
- いくつかのルーチンで、変数をconst受け取るが、呼び出した先のルーチンで変数に代入してる場合があった(未issue)。
- WindowsはMINGW64でもLLP64であった。MPFR版で複素数の割り算で問題に
 - 複素数の割り算はSmithのidiomを使うのが通例; 分母がover/underflowしやすいため
Robert L. Smith. Algorithm 116: Complex division. Commun. ACM, 5(8):435, 1962.
 - LLP64; Windows int = 4bytes, long int = 4 bytes, long long = 8bytes
 - LP64: Linux/macOS = 4bytes , long int = 8 bytes
 - GMPの内部は変わらず→GMPの入出力はlong intのみ→MPFRの指数部の内部表現はlong int; 実際に使えるのはint→LLP64でMPFRの指数部は余裕なし→MPCの割り算は高速化のためidiomを使わない→LLP64で指数部long int = int なので overflow, underflowしやすくなる(未issue)

$$z = \frac{ac + bd}{c^2 + d^2} + i \frac{bc - ad}{c^2 + d^2}$$

LAPACKのTESTINGの移植

- 対応すべきxxx.inの数は28個ある。
- feasibleな労力でできた。
- 2年かけて、実、複素(非対称固有値問題、特異値問題)が解けるようになった。

dbak.in	dgbal.in	dtest.in	zec.in	zsb.in
dbal.in	dgd.in	dtest_rfp.in	zed.in	zsg.in
dbb.in	dgg.in	zbak.in	zgbak.in	ztest.in
dec.in	dsb.in	zbal.in	zgbal.in	ztest_rfp.in
ded.in	dsg.in	zbb.in	zgd.in	
dgbak.in	dstest.in	zctest.in	zgg.in	

MPLAPACKのQA風景

- 2年頑張ると...

```
Cbak.in      Ced.in      Csb.in      Rbb.in      Rgd.in      glm.in      se2.in      xeigtstC__Float128  xeigtstC_mpfr  xeigtstR_doubl
Cbal.in      Cgbak.in     Csg.in      Rec.in      Rgg.in      gqr.in      sep.in      xeigtstC__Float64x  xeigtstC_qd    xeigtstR_gmp
Cbal_double.in Cgbal.in    Rbak.in     Red.in      Rsb.in      gsv.in      svd.in      xeigtstC_dd        xeigtstR__Float128  xeigtstR_mpfr
Cbb.in       Cgd.in      Rbal.in     Rgbak.in    Rsg.in      lse.in      test_eig_all.sh  xeigtstC_double    xeigtstR__Float64x  xeigtstR_qd
Cec.in       Cgg.in      Rbal_double.in Rgbal.in    csd.in      nep.in      test_eig_all_mingw.sh xeigtstC_gmp       xeigtstR_dd
```

```
test.in      test_lin_all_mingw.sh  xlintstC_gmp      xlintstR_dd      xlintstrfC__Float128  xlintstrfC_mpfr      xlintstrfR_double
test_rfp.in  xlintstC__Float128    xlintstC_mpfr     xlintstR_double  xlintstrfC__Float64x  xlintstrfC_qd        xlintstrfR_gmp
Rtest.in     xlintstC__Float64x    xlintstC_qd       xlintstR_gmp     xlintstrfC_dd        xlintstrfR__Float128  xlintstrfR_mpfr
Rtest_rfp.in xlintstC_dd          xlintstR__Float128 xlintstR_mpfr    xlintstrfC_double    xlintstrfR__Float64x  xlintstrfR_qd
test_lin_all.sh xlintstC_double      xlintstR__Float64x xlintstR_qd      xlintstrfC_gmp       xlintstrfR_dd
```

全7精度で、QA(テスト)プログラムがビルドできた
さらにテストを通すということもできた

MPLAPACKのQA風景

binary128での線形ルーチンQA風景

```
Tests of the Multiple precision version of LAPACK MPLAPACK VERSION 2.0
Based on the original LAPACK VERSION 3.10.1

The following parameter values will be used:
  M :    0    1    2    3    5   10   50
  N :    0    1    2    3    5   10   50
 NRHS :    1    2   15
  NB :    1    3    3    3   20
  NX :    1    0    5    9    1
 RANK :   30   50   90

Routines pass computational tests if test ratio is less than 30.00

Relative machine underflow is taken to be : +3.3621031431120935e-4932
Relative machine overflow is taken to be : +1.1897314953572318e+4932
Relative machine precision is taken to be : +1.9259299443872359e-34
RGE routines passed the tests of the error exits

All tests for RGE routines passed the threshold ( 3653 tests run)
RGE drivers passed the tests of the error exits

All tests for RGE drivers passed the threshold ( 5748 tests run)
RGB routines passed the tests of the error exits
```

MPFRでのSVD QA風景

```
Tests of the Singular Value Decomposition routines
Tests of the Multiple precision version of LAPACK MPLAPACK VERSION 2.0.1
Based on original LAPACK VERSION 3.10.1

The following parameter values will be used:
  M:    0    0    0    1    1    1    2    2    3    3
      3   10   10   16   16   30   30   40   40
  N:    0    1    3    0    1    2    0    1    0    1
      3   10   16   10   16   30   40   30   40
 NB:    1    3    3    3    20
NBMIN:  2    2    2    2    2
  NX:    1    0    5    9    1
  NS:    2    0    2    2    2

Relative machine underflow is taken to be+9.5302596195518043e-323228497
Relative machine overflow is taken to be+2.0985787164673877e+323228496
Relative machine precision is taken to be+7.4583407312002067e-155

Routines pass computational tests if test ratio is less than+5.0000000000000000e+01

ZBD routines passed the tests of the error exits ( 35 tests done)
Cgesvd passed the tests of the error exits ( 8 tests done)
Cgesdd passed the tests of the error exits ( 6 tests done)
Cgejsv passed the tests of the error exits ( 11 tests done)
Cgesvdx passed the tests of the error exits ( 12 tests done)
Cgesvdq passed the tests of the error exits ( 11 tests done)

SVD:  NB =  1, NBMIN =  2, NX =  1, NRHS =  2

All tests for CBD routines passed the threshold ( 4085 tests run)
```

- テストはほぼ通る。
- すべては安定してテストに通らない→乱数を真面目に振ってるから
- ひたすら時間がかかる!! + 7つの精度ごとすべてQAに通さなくてはならない

<https://github.com/nakatamaho/mplapack/tree/master/mplapack/test/lin/results>

<https://github.com/nakatamaho/mplapack/tree/master/mplapack/test/eig/results>

昔々:MPACK 0.8.0 (2012-12-25)

- 多倍長精度のBLAS, LAPACK
- API提供を目的
- MBLASはすべて実装、MPALACPKは100ルーチン実装。
 - 実対称、エルミート固有値問題、LU, コレスキー分解、逆行列、条件数推定対応
- Linux/BSD/Mac/Win対応
- GMP, MPFR, binary128, quad-double, double-double, double, long double 対応
 - Rgemm (double-double) はCUDAでC2050対応
 - Intel Xeon Phi対応
- C++で書かれていて、double/floatのようにプログラム可能。
- 2-条項BSDライセンス

MPLAPACK 2.0.1 (2022-09-12)

Lower triangular case

Upper triangular case

$$\begin{pmatrix} 1 & & & & & & \\ 2 & 8 & & & & & \\ 3 & 9 & 14 & & & & \\ 4 & 10 & 15 & 19 & & & \\ 5 & 11 & 16 & 20 & 23 & & \\ 6 & 12 & 17 & 21 & 24 & 26 & \\ 7 & 13 & 18 & 22 & 25 & 27 & 28 \end{pmatrix} \quad \begin{pmatrix} 1 & 2 & 4 & 7 & 11 & 16 & 22 \\ & 3 & 5 & 8 & 12 & 17 & 23 \\ & & 6 & 9 & 13 & 18 & 24 \\ & & & 10 & 14 & 19 & 25 \\ & & & & 15 & 20 & 26 \\ & & & & & 21 & 27 \\ & & & & & & 28 \end{pmatrix}$$

- 多倍長精度のBLAS, LAPACK
- LAPACK 3.9.2 をベースにした。
- MPLAPACKほぼ実装 実+複素
 - 実対称、エルミート固有値問題、LU, コレスキー分解、特異値
 - 一般化固有値問題、最小二乗法、逆行列、条件数推定対応
 - RFP (rectangular full packed) format対応
 - 未対応: 混合精度
- Linux/Mac/Win対応
- GMP, MPFR, binary128 (_Float128), quad-double, double-double, double, _Float64x 対応
 - Rgemm (double-double) はCUDAでA100, V100, H100対応
- C++で書かれていて、double/floatのようにプログラム可能。
- 2-条項BSDライセンス

RFP format

MPLAPACKを使ってみる

試し方

- <https://github.com/nakatamaho/mplapack> を見てね
Dockerで試せます

Ubuntu, CentOS amd64(x86_64)/intel one API, aarch64, win64
macOSではmacportsかbrewなどで

Ubuntu 22.04

```
$ git clone https://github.com/nakatamaho/mplapack/  
$ cd mplapack  
$ /usr/bin/time docker build -t mplapack:ubuntu2204 -f Dockerfile_ubuntu22.04 . 2>&1 | tee log.ubuntu
```

```
$ sudo port install gcc10 coreutils git gsed  
$ rm -rf $HOME/tmp $HOME/MPLAPACK  
$ mkdir $HOME/tmp  
$ cd $HOME/tmp  
$ wget https://github.com/nakatamaho/mplapack/releases/download/v2.0.1/mplapack-2.0.1.tar.xz  
$ tar xvfz mplapack-2.0.1.tar.xz  
$ cd mplapack-2.0.1  
$ CXX="g++-mp-10" ; export CXX  
$ CC="gcc-mp-10" ; export CC  
$ FC="gfortran-mp-10"; export FC  
$ INSTALL="/usr/bin/install"; export INSTALL  
$ ./configure --prefix=$HOME/MPLAPACK --enable-gmp=yes --enable-mpfr=yes --enable-_Float128=yes --enable-  
$ make -j4  
$ make install
```

サンプルプログラム

- ビルドが終わると、`${prefix}/share/examples/mplapack`ディレクトリ以下に例が沢山できる。いくつか例をあげると…
- `00_LinearEquations/*getri_test*`
 - MPFR/GMP/QD...などで任意精度で実、複素数行列逆行列を求める。
- `00_LinearEquations/Rgetri_Hilbert_XXX`
 - MPFR/GMP/QD...などでヒルベルト行列とその逆行列を求める。
- `04_NonsymmetricEigenproblems/*geev*`
 - MPFR/GMP/QD...などで非対称行列の固有値問題を解く
- `05_SingularValueDecomposition/*gesvd_test_*`
 - MPFR/GMP/QD...などで特異値問題を解く

Rgemmのベンチマーク

dgemmの多倍長精度版はRgemm

実装、テスト、パフォーマンスをみてよう

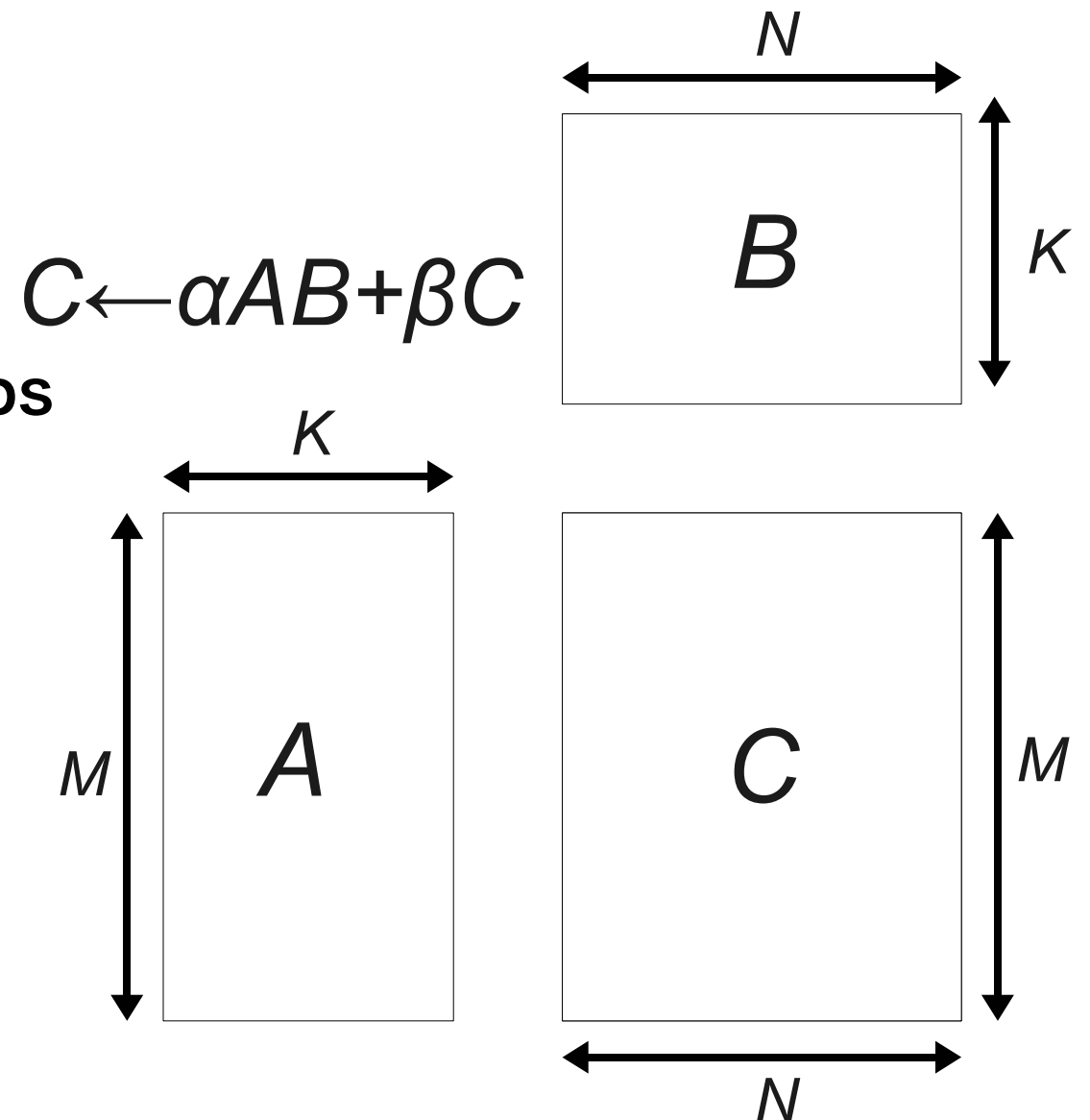
GMP版512bit(153桁): 600MFlops

Binary128(33桁) : 2000Mflops

Double-double(32桁): 18GFlops-550GFlops

Ryzen 3970X(32cores), NVIDIA A100

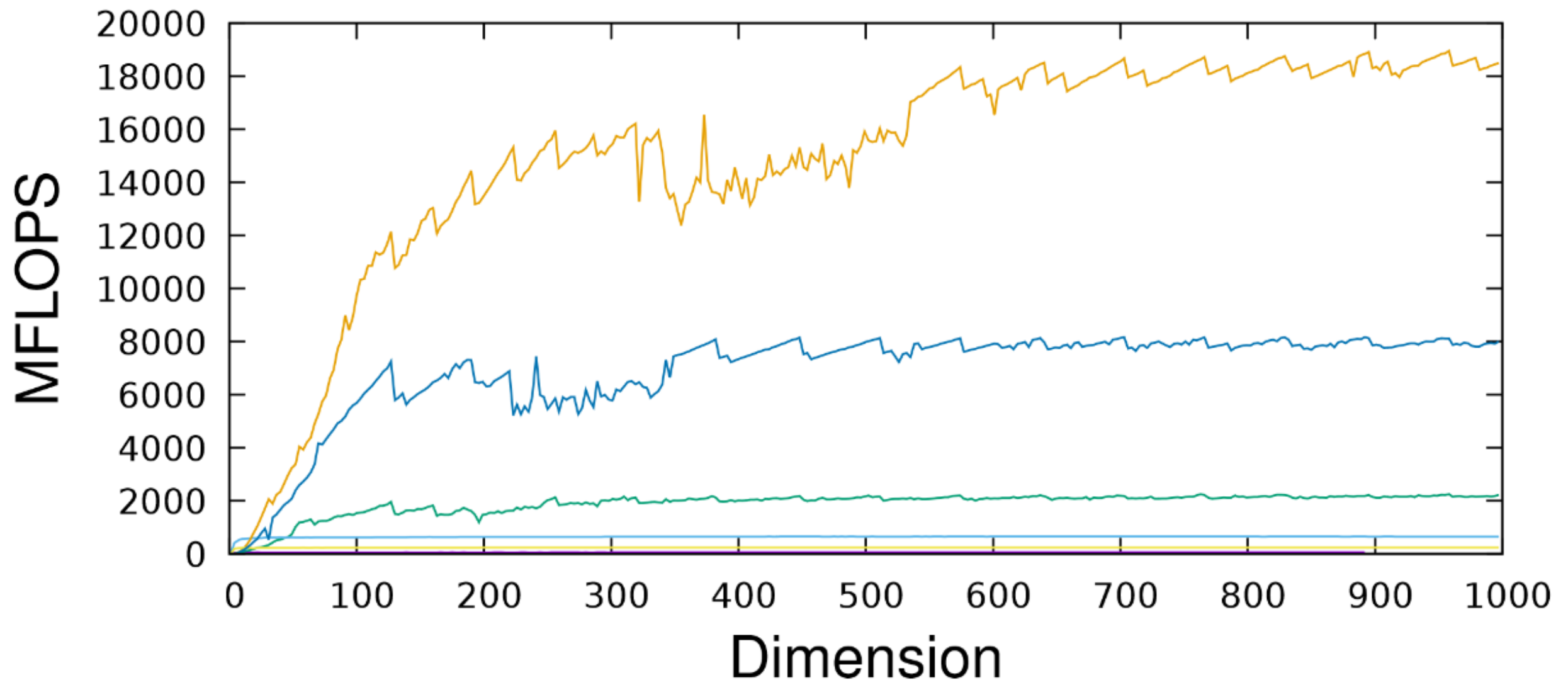
double-double計算はA100で実用になる



A benchmark of Rgemm

Float128 _Float64x (OpenMP)
_Float128 (OpenMP) double-double
_Float64x double-double (OpenMP)

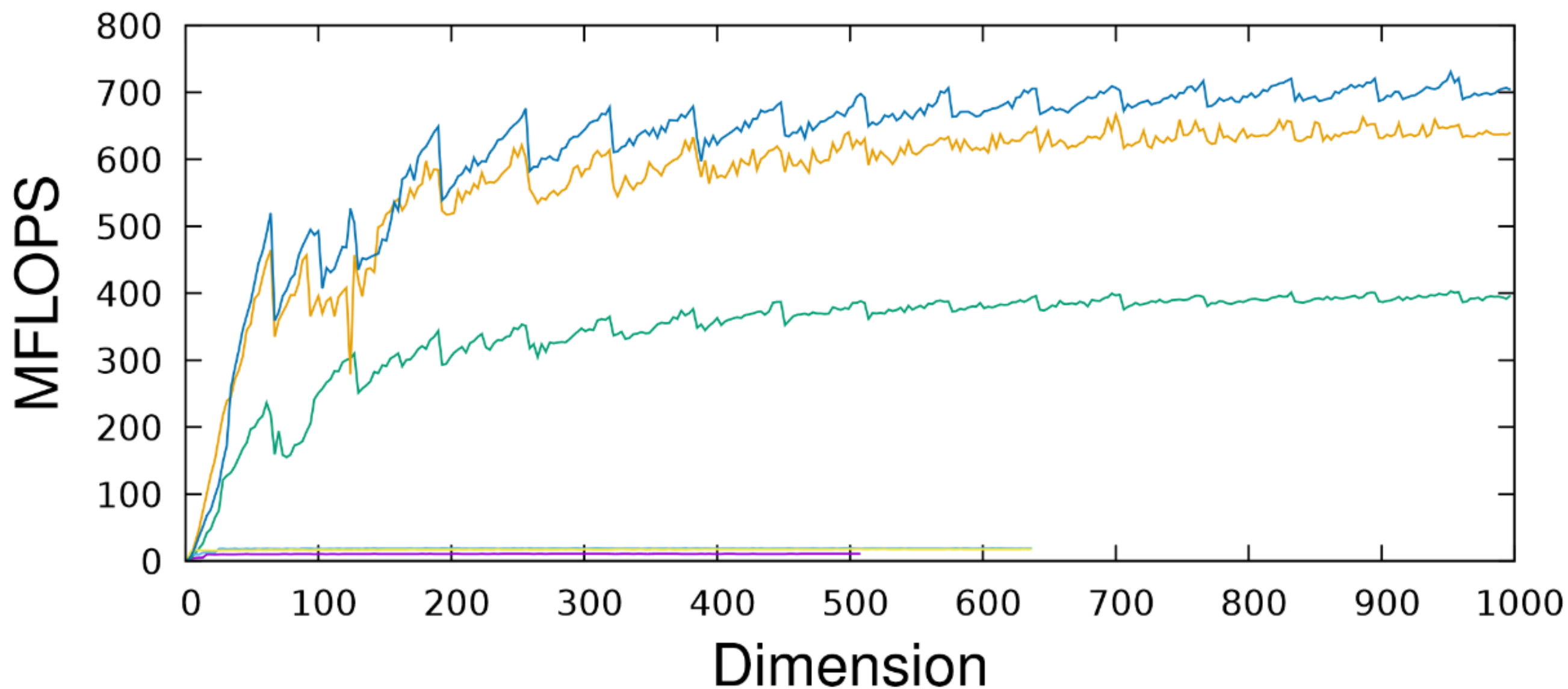
Rgemm on AMD Ryzen Threadripper 3970X 32-Core Proce



A benchmark of Rgemm

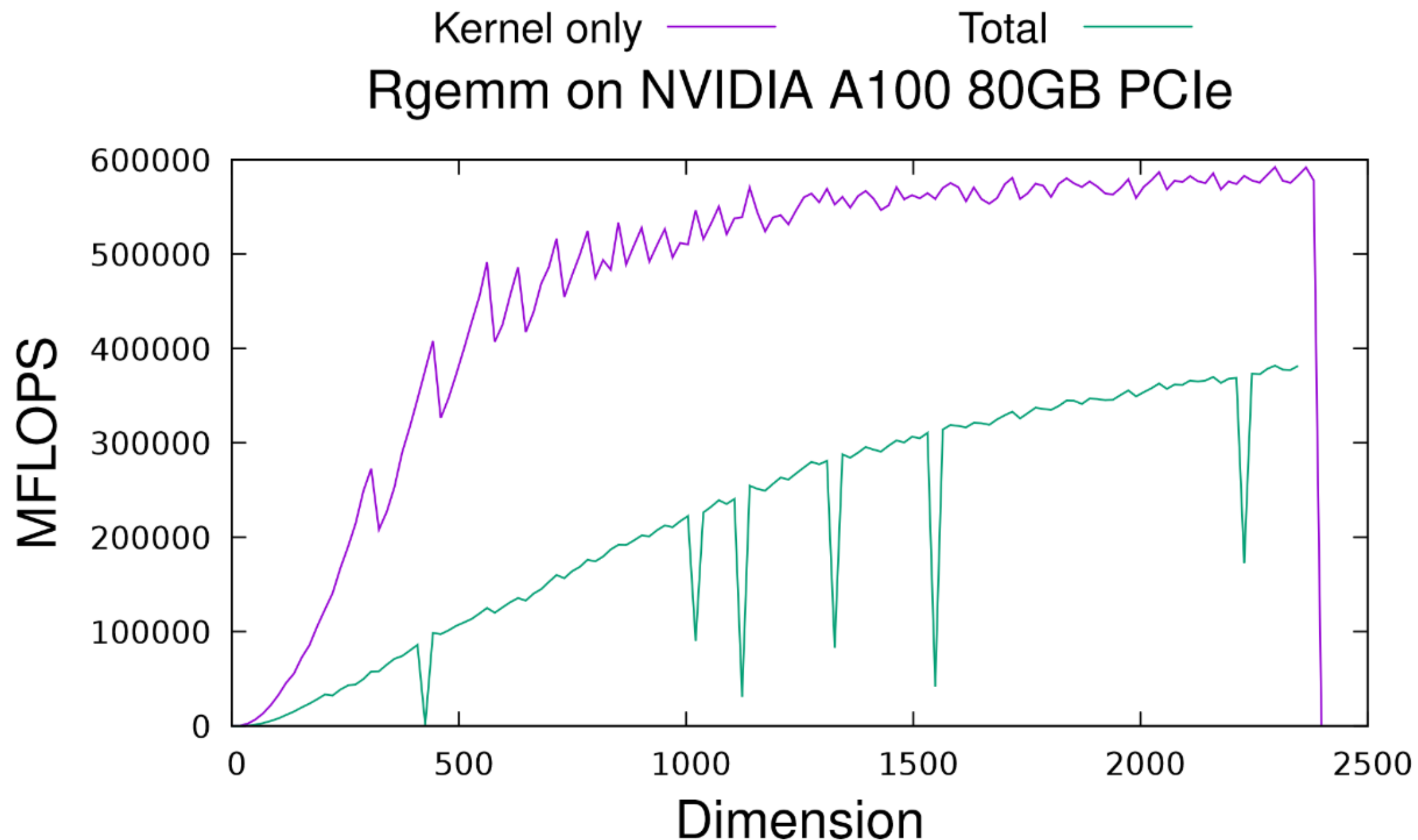
MPFR 512bit — GMP 512bit(OpenMP) —
MPFR 512bit(OpenMP) — quad-double —
GMP 512bit — quad-double(OpenMP) —

Rgemm on AMD Ryzen Threadripper 3970X 32-Core Proces



double-doubleでのRgemm on A100

ほぼ四倍精度、まさかの500GFlops超え！



他のベンチマーク結果

  master [mplapack](#) / [benchmark](#) / [results](#) / 2022 / 

 nakatamaho .

Name	Last commit message
 ..	
 aarch64-unknown-linux-gnu-raspberry-pi4	.
 x86_64-apple-darwin20.6.0-Corei5-8500B	.
 x86_64-pc-linux-gnu-Ryzen-Threadripper-3970X-RTX3080	.
 x86_64-pc-linux-gnu-Ryzen-Threadripper-3970X-TeslaA100	.
 x86_64-pc-linux-gnu-Ryzen-Threadripper-3970X-inteloneapi	.
 x86_64-pc-linux-gnu-Xeon-E5-2623-TeslaV100	.

<https://github.com/nakatamaho/mplapack/tree/master/benchmark/results/2022>

MPLAPACK TODO

This is the release process for MPLAPACK 3.0.0

Action	Date	Status	Description
optimized implementations should be the default			
add template version			mockup https://github.com/nakatamaho/mplapack-template
add gmpfrxx			https://math.berkeley.edu/~wilken/code/gmpfrxx/
add openblas for benchmark of double			
update to LAPACK 3.10.1			
FMA for QD, DD			
add more benchmarks, Rsyev, Rgesvd etc			
optimized implementations			
add qa program of BLAS			
Take benchmark on A100 (Rgemm, Rsyrk dd)			
python integration?			
octave integration?			
Drop GMP version			Since trigonometric functions req'd
Impliment mixed precision version			
cleanup pow (REAL, long int)			
Get rid of compiler warnings			
lp64 ilp64 llp64 ilp32 lp32 cleanup			

ソフトウェアで学んだこと

- 中田からは何もしていない。待っているだけであった。
- Fable: FORTRANからC++へ、可読性のあるコンバータの登場。
 - Pythonで書かれているため、パーサへの手入れが簡単。
 - 2012年頃はpython知らなかった。2015年ころから使い始めた。
 - FEM (Fortran EMulator) の搭載：**write, formatがほぼそのまま使えた。**
- Clang-format: C++の整形ツール。sedに渡しやすい。
- Docker: OSやアーキテクチャを手軽に試せた。
- Github: webベースの存在ありがたい。
- デパス: 鈍い頭痛、肩こりに20年位悩まされていた。
- コロナ: 通勤しなくて良くなった。

ソフト開発で学んだこと+感謝

- 線形代数は人類永遠の課題。
 - どこにでも出てくる。
 - いつでも何か課題が潤沢にある。理論的？計算機的？HPC的？
 - 人間の頭では非線形を理解出来ないのかもしれない。
- プログラムを書くことは**生への執念、命を燃やす、命を削ること**。
 - なるべく自分を追い詰め過ぎず、適度に甘やかすこと
 - 燃え尽き症候群となると回復に時間がかかった。工夫が必要。
 - 体調管理。デパスが頭痛、肩こりを随分和らげてくれた。
 - 道具がないときは作るじゃなくて待つのも良からう。
 - モチベーションを維持できないときは出てくるまで待てば良い。
 - 最大限樂をするために最大限努力せよ(これはよく言われる)。
- 姫野先生に拾ってもらった(多謝)
 - 失業せず何とかなった。8年の停滞がありながらも続けることができた。
- 下司先生に感謝
 - **前回の授業や本執筆でMPLAPACK作成のモチベーションもらった。**
- 書ききれないほどたくさんの人に暖かい言葉や支援をいただいた。